



Ciclo Formativo de Grado Superior **Desarrollo de Aplicaciones Multiplataforma**

NEURO-FOCUS

Curso 2025-26

Autor: ISAAC Sánchez García

Tutor: JUAN Bonnín



ÍNDICE

1. Origen y justificación del proyecto.....	1
1.1. Contextualización: La brecha digital en personas con movilidad reducida.	1
1.2. Motivación personal: El reto de la accesibilidad universal.	1
1.3. Justificación técnica: Por qué una solución <i>Low-Cost</i> frente a equipos médicos.	2
2. Introducción.....	3
2.1. Concepto de NEURO-FOCUS: Interacción híbrida Ojo-Cerebro.....	3
2.2. Estado del arte: Evolución del <i>Eye-Tracking</i> y las interfaces BCI.	4
2.3. Análisis de soluciones competidoras.	5
3. Objetivos	7
3.1. Objetivo general: Control total del sistema operativo sin manos.....	7
3.2. Objetivos específicos:	7
3.2.1. Minimizar la latencia en el clic.....	7
3.2.2. Garantizar la robustez frente a movimientos oculares involuntarios. ..	7
3.2.3. Dotar de autonomía del usuario mediante herramientas integradas. ..	8
4. Tecnología usada en el proyecto	9
4.1. Lenguaje de programación: Python y su ecosistema.	9
4.2. Visión artificial: MediaPipe Face Landmarker y OpenCV.	9
4.3. Interfaz de usuario (GUI): CustomTkinter.	10
4.4. Hardware y biosensores: Diadema NeuroSky MindWave Mobile 2.	12
4.5. Librerías auxiliares: PyAutoGUI, PySerial y NumPy.	13
5. Metodología y desarrollo del proyecto.....	15
5.1. Metodología de desarrollo ágil: Sprints y prototipado rápido.	15
5.2. Planificación temporal: Cronograma y Diagrama de Gantt.	16
5.3. Diseño de la Arquitectura: Sistema multihilo (Visión, Diadema y GUI). .	17

5.4. El Motor Visual:	19
5.4.1. Algoritmo de calibración dinámica de 9 puntos y extracción de marcadores faciales.	19
5.4.2. Implementación del Filtro de Kalman para la estabilización del cursor.	22
5.5. El Motor de Clic:	24
5.5.1. Procesamiento de señales crudas de la diadema.	24
5.5.2. Lógica del "Gatillo Fácil" basada en calidad de señal.	28
5.6. Ingeniería de Usabilidad (UX):	31
5.6.1. Sistema de Freno de Mano inteligente.....	31
5.6.2. Mecanismo de "Rollback Temporal" (Máquina del tiempo).....	34
5.7. Integración de herramientas: Teclado virtual y dictado por voz.....	37
5.8. Despliegue: Generación y empaquetado del ejecutable (.exe).....	40
6. Medios y cálculo de costes	43
6.1. Recursos materiales y hardware.....	43
6.2. Recursos de software y licencias.	43
6.3. Costes de personal e ingeniería (Cálculo de horas/hombre).	44
6.4. Presupuesto total del proyecto.....	45
6.5. Modelo de negocio y viabilidad económica	46
7. Conclusiones y problemas confrontados	47
7.1. Análisis de resultados: Precisión y tiempo de respuesta.	47
7.2. Problemas técnicos superados: Ruido lumínico y desviación palpebral... 47	
7.3. Desviaciones del anteproyecto:.....	48
7.3.1. Pivotaje de ondas EEG a señales musculares (EMG) por latencia.	48
7.3.2. Descarte de la versión móvil por densidad de píxeles.	49
7.4. Valoración final del cumplimiento de objetivos.	50
8. Mejoras y trabajos futuros	53
8.1. Escalabilidad física: Control de silla de ruedas mediante Arduino.....	53

8.2. Evolución académica: Integración de Machine Learning avanzado (Máster en Tajamar).....	53
8.3. Aplicación de IA para el análisis de fatiga y atención del usuario.	54
9. Bibliografía.....	55
10. Anexos	57
10.2. Glosario de términos técnicos.	57
10.2. Fragmentos de código fuente críticos.	59
10.2.1. Mapeo de coordenadas mediante mínimos cuadrados.....	59
10.2.2. Sistema de Rollback y estabilización mediante Filtro de Kalman ...	60
10.2.3. Decodificación a bajo nivel de la telemetría Bluetooth	62
Agradecimientos.....	67



1. Origen y justificación del proyecto

1.1. Contextualización: La brecha digital en personas con movilidad reducida.

En la actualidad, se vive en la era de la información, el uso de un equipo informático se convierte en algo prácticamente indispensable para tareas muy diversas, como pueden ser el desarrollo personal, educativo o profesional, no obstante, este modelo se ve condicionado por una interacción tradicional con los sistemas operativos, diseñados bajo el estándar de Dispositivos de Interfaz Humana (HID). La dependencia de periféricos físicos como el teclado y el ratón plantea una limitación importante, ya que este ecosistema de *hardware* asume que el usuario posee una capacidad motriz fina e intacta en sus extremidades superiores.

Para un sector significativo de la población que padece de discapacidades motoras severas, tales como la Esclerosis Lateral Amiotrófica (ELA), tetraplejia, lesiones en la médula o parálisis cerebral, el uso de estos periféricos tradicionales resulta físicamente imposible, lo cual genera una enorme brecha digital, produciéndoles un gran aislamiento y privación en tareas básicas de comunicación o acceso a la información. Todo ello afecta gravemente a sus oportunidades laborales y limita drásticamente su autonomía.

1.2. Motivación personal: El reto de la accesibilidad universal.

El presente proyecto nace de la necesidad de aplicar los conocimientos adquiridos en el ciclo "Desarrollo de Aplicaciones Multiplataforma" para resolver un problema de impacto social real a nivel mundial. La motivación principal reside en el reto técnico de concebir, diseñar y programar una herramienta de accesibilidad universal que le permita a cualquier persona afectada por una limitación motora (de cuello hacia abajo), independientemente de su nivel socioeconómico, recuperar una autonomía real frente al ordenador.

Por ello se plantea el desarrollo de NEURO-FOCUS, un sistema de software híbrido que es capaz de traducir los micromovimientos oculares y señales biomusculares o cerebrales en comandos nativos del sistema operativo. Esto permite una navegación fluida e intuitiva, asegurando el control del equipo sin requerir ningún tipo de interacción manual.

1.3. Justificación técnica: Por qué una solución *Low-Cost* frente a equipos médicos.

En la actualidad, el mercado de las tecnologías asistenciales ofrece soluciones funcionales de seguimiento ocular (*eye-tracking*) y sistemas BCI (*Brain-Computer Interface*). Sin embargo, estas alternativas comerciales suelen ser dispositivos de grado médico o de *hardware* propietarios, como los de las marcas líderes del sector, cuyos precios rondan los varios miles de euros. Este coste prohibitivo las convierte en un artículo de lujo, el cual resulta inaccesible para la mayoría de los pacientes y sus familias, que normalmente afrontan elevados gastos derivados por su condición médica.

La justificación fundamental de NEURO-FOCUS radica en la democratización de la tecnología de asistencia. A nivel de ingeniería de software, el proyecto demuestra que es posible alcanzar niveles de precisión y usabilidad, comparables a los de dispositivos médicos de alto coste, utilizando únicamente hardware comercial asequible. Mediante la integración de una cámara web estándar de 60 FPS o superior, una diadema Bluetooth de lectura de ondas EEG/EMG (*NeuroSky MindWave Mobile 2*) y el procesamiento avanzado de visión artificial con Python, se ha desarrollado una solución *open source* de bajo presupuesto. Este enfoque rompe la barrera económica de la accesibilidad digital y demuestra la viabilidad de la innovación de bajo coste, en sintonía con otros grandes hitos históricos de la inclusión, como lo fue en su momento el desarrollo de prótesis con piezas de LEGO.

2. Introducción

2.1. Concepto de NEURO-FOCUS: Interacción híbrida Ojo-Cerebro.

El proyecto de NEURO-FOCUS se puede definir como una solución de *software* de asistencia diseñada para el ecosistema de *Windows*, cuyo objetivo principal es permitir un control completo del ordenador mediante el uso de una interfaz de interacción híbrida. A diferencia de sistemas de accesibilidad clásicos que dependen de una única fuente de entrada, este sistema combina dos tecnologías de vanguardia para segmentar en dos las funciones de navegación y ejecución.

En primer lugar, se incorpora un motor de *eye-tracking* (seguimiento ocular), basado en la visión artificial mediante el uso de una *webcam* estándar como sensor de entrada de vídeo, con esa información que envía la *webcam*, el *software* en tiempo real procesa la posición de las pupilas del usuario para mapear la mirada sobre las coordenadas de la pantalla, permitiendo un control del desplazamiento del cursor de forma fluida, para lograr la máxima precisión en el control del cursor, es necesario realizar un calibrado lo más exacto posible.

En segundo lugar, para la ejecución de las acciones, NEURO-FOCUS integra una interfaz cerebro ordenador (*Brain-computer interface* o *BCI*) mediante una conexión vía *Bluetooth*, usando una diadema *NeuroSky MindWave Mobile 2*; El sistema monitoriza las señales bioeléctricas del usuario y el *software* que está optimizado para identificar un patrón concreto de contracción muscular, en nuestro caso, fruncir el ceño o parpadeo voluntario. Tras la identificación del patrón, se traduce de forma instantánea en un evento de clic principal. Esta mecánica, denominada «gatillo fácil», permite al usuario interactuar con la totalidad de la interfaz de *Windows* sin necesidad de realizar movimientos físicos complejos o fatigosos. El sistema simplifica la navegación al sustituir el doble clic nativo del sistema operativo por una acción de un único clic. De este modo, para ejecutar cualquier aplicación, el usuario solo realizará un parpadeo voluntario.

Para garantizar la autonomía total y suplir la falta de interacción manual, la aplicación integra herramientas de apoyo estratégico tales como el despliegue de un teclado virtual para la entrada de texto y el acceso al sistema de dictado por voz nativo. De esta forma, NEURO-FOCUS se presenta como una estación de accesibilidad completa donde el usuario, mediante la combinación de la mirada y un gesto muscular (parpadeo), puede navegar por su equipo, comunicarse o acceder a internet dotando al usuario de un nuevo nivel de autonomía.

2.2. Estado del arte: Evolución del *Eye-Tracking* y las interfaces BCI.

El seguimiento ocular (*eye-tracking*) ha sufrido una enorme evolución en la última década. Tradicionalmente, esta tecnología dependía de un *hardware* especializado que resultaba muy costoso. Se utilizaba un dispositivo que mediante el uso de luces infrarrojas y cámaras de alta velocidad reflejaban luz en la córnea del usuario (usado, por ejemplo, en los equipos de Tobii).

Pero, en la última década, gracias a los avances en inteligencia artificial y redes neuronales, estamos en un momento en el que el estado del arte nos permite realizar un seguimiento preciso utilizando únicamente una cámara web y el uso de herramientas de aprendizaje automático (*Machine learning*), como es *Media Pipe* de *Google*, que es capaz de mapear cientos de puntos faciales en tiempo real y poder llegar a calcular la dirección de la mirada sin necesidad de sensores infrarrojos.

Por otra parte, las interfaces Cerebro-Ordenador (*Brain-Computer Interfaces* o BCI) también han dado un salto importante, ya que han salido del uso médico hacia el consumidor general. Antiguamente, leer señales biomédicas requería de un gorro con decenas de electrodos húmedos y geles conductores, en cambio, en la actualidad, existen sensores secos y portables como la diadema *NeuroSky*, empleada en este proyecto, *que se conectan por bluetooth*. Estas diademas normalmente tienen la capacidad de leer dos cosas, las ondas EEG o actividad cerebral pura, como son la concentración o relajación y las señales EMG o la actividad electromuscular, que se genera al mover los músculos de la cara en acciones como el parpadeo o fruncir el ceño.

El motivo principal por el que las ondas EEG han sido descartadas como método de ejecución directa se debe a que, dentro del estado del arte enfocado a la accesibilidad interactiva, la latencia es un factor crítico; Las señales EMG han demostrado poseer una velocidad de propagación mucho más rápida y ser significativamente más fiables que las ondas cerebrales para ejecutar comandos inmediatos, como es el caso de un clic de ratón.

2.3. Análisis de soluciones competidoras.

En el mercado de la tecnología asistencial basada en el seguimiento visual existen soluciones altamente consolidadas. Empresas referentes en este sector, como **Tobii Dynavox** o **Irisbond**, ofrecen sistemas de seguimiento ocular de alta precisión que permiten la interacción con los equipos informáticos. No obstante, el modelo de desarrollo de estas alternativas competidoras se basa en ecosistemas cerrados y hardware propietario. Esta dependencia obliga al usuario a adquirir equipamiento específico, como monitores de grado médico o barras de sensores infrarrojos, lo que eleva el coste final de las soluciones hasta cifras que resultan prohibitivas para el usuario medio.

Por otra parte, el objetivo de NEURO-FOCUS es opuesto a esto, y se desmarca de la dependencia de tecnología exclusiva; Se plantea como una solución fundamentada en el procesamiento de *software* que se apoya de manera directa en el uso de periféricos comerciales de bajo costo como la *webcam* o la diadema *bluetooth*.

El proyecto se posiciona como una alternativa competitiva enfocada en términos de accesibilidad económica y versatilidad, ya que las soluciones presentes en el mercado obligan al usuario a realizar un desembolso económico alto para adaptar su entorno. NEURO-FOCUS utiliza la inteligencia artificial para adaptar el software al equipo informático ya existente del usuario, eliminando la barrera de entrada a la tecnología asistencial.

3. Objetivos

3.1. Objetivo general: Control total del sistema operativo sin manos.

El objetivo principal de este proyecto es el diseño y desarrollo de un software de accesibilidad funcional que permita a personas con diversidad funcional motora severa controlar el sistema operativo *Windows* de forma autónoma. Para lograrlo, se implementa una interfaz basada en el seguimiento ocular para el desplazamiento del cursor y en la lectura de señales electromusculares (mediante el parpadeo voluntario) para la ejecución del clic, utilizando exclusivamente *hardware* comercial de bajo coste.

3.2. Objetivos específicos:

Para asegurar la viabilidad y usabilidad del sistema y cumplir con el objetivo general se han marcado una serie de objetivos secundarios .

3.2.1. Minimizar la latencia en el clic.

Lograr que la lectura de la señal bioeléctrica mediante la diadema y su posterior traducción en un evento del ratón se realice en el menor tiempo posible.

En el ámbito de la accesibilidad tecnológica, una alta latencia entre la orden que ejecuta el usuario y la acción de ésta en la pantalla genera frustración, fatiga, y dificulta el control, por lo que el objetivo principal no sería posible. Por ello, se establece como objetivo optimizar el bucle de procesamiento del *software* y la utilización de las señales musculares sobre las cerebrales para asegurar una respuesta prácticamente instantánea.

3.2.2. Garantizar la robustez frente a movimientos oculares involuntarios.

Desarrollar un sistema de estabilización del cursor que elimine el temblor característico de los sistemas de visión artificial y prevenga los clics erróneos. Para ello, se plantea la integración de algoritmos matemáticos predictivos (como el Filtro de Kalman) para suavizar la trayectoria visual. Asimismo, se fija el objetivo de implementar un mecanismo de freno y retroceso temporal (*rollback*), capaz de ignorar las microdesviaciones bruscas de la pupila que ocurren de forma

natural y biomecánica en el instante exacto en el que el usuario contrae los músculos faciales para ejecutar el clic.

3.2.3. Dotar de autonomía del usuario mediante herramientas integradas.

Hay que asegurar que el usuario no dependa de la asistencia de un tercero para realizar tareas informáticas cotidianas como consultar un periódico o enviar un correo. Este objetivo implica la creación y diseño de una interfaz gráfica de usuario lo más minimalista posible y no intrusiva que permita el acceso a funciones o herramientas de accesibilidad nativas del sistema operativo, y a su vez, simplificando las funciones críticas como el despliegue de teclado virtual en pantalla o la activación del dictado por voz , permitiendo una reducción considerable del esfuerzo físico y la carga cognitiva para navegar y comunicarse.

4. Tecnología usada en el proyecto

4.1. Lenguaje de programación: Python y su ecosistema.

El lenguaje de programación principal seleccionado para el desarrollo del núcleo de NEURO-FOCUS ha sido *Python*[6], la elección de esta tecnología no se ha producido de manera aleatoria, sino que, responde a que, en el estado del arte actual, *Python* está consolidada como el estándar industrial actual para el desarrollo de aplicaciones de inteligencia artificial, *machine learning* y el procesamiento de señales en tiempo real.

Una de las principales justificaciones del uso de este lenguaje es la capacidad que dispone para actuar como un lenguaje aglutinador, esto permite que el *software* pueda orquestar e integrar librerías mucho más complejas, escritas en otros lenguajes de bajo nivel tales como C o C++(por ejemplo, *OpenCV*), manteniendo una sintaxis limpia y legible.

Además, la arquitectura de este proyecto requería obligatoriamente un control de concurrencia ya que NEURO-FOCUS debe realizar múltiples tareas críticas de forma simultánea tales como, capturar el video a una tasa de fotogramas alta (60fps o superior), recibir los paquetes de telemetría bioeléctrica, que son enviados por la diadema vía *bluetooth* a una frecuencia de 512Hz, aplicar filtros matemáticos y al mismo tiempo renderizar una imagen gráfica sin que el sistema operativo detecte que la aplicación no responde. Para dar solución a esto, *Python* permite implementar una arquitectura multihilo (*multithreading*) usando *threading*, el cual es un módulo nativo del lenguaje. De esta forma los procesos de visión, lectura del *hardware* e interfaz gráfica de usuarios (GUI) han sido segmentados en hilos de ejecución paralelos y asíncronos, lo que garantiza un flujo continuo de datos, evitando congelaciones de la aplicación y asegurando una latencia casi nula.

4.2. Visión artificial: MediaPipe Face Landmarker y OpenCV.

El sistema encargado de transformar la mirada del usuario en coordenadas en la pantalla se basa en la combinación de dos de las herramientas más potentes del mercado tecnológico actual, siendo *OpenCv* y el ecosistema de *MediaPipe*[2].

Por una parte, se utiliza *OpenCv*, (*Open Source Computer Vision Library*) que actúa como el motor que se encarga de obtener el vídeo, siendo la responsable

de establecer una conexión de bajo nivel con la *webcam* del usuario, gestionar el buffer de memoria para evitar retrasos de cola (*queue lag*) y extraer los fotogramas de manera individual; Un aspecto técnico crítico que resuelve OpenCV es la transformación matricial de las imágenes en estructuras de datos multidimensionales (*arrays*) mediante la librería NumPy, así como la conversión del espacio de color. Por defecto, el flujo de vídeo es capturado en el formato BGR (*Blue, Green, Red*), sin embargo, los modelos de redes neuronales requieren una entrada en formato RGB (*Red, Green, Blue*). Esta conversión se ejecuta en el orden de los microsegundos, garantizando la adecuación de la matriz antes de ser inyectada en el modelo predictivo.

Por otra parte, el análisis de la mirada se delega en MediaPipe, una solución *open source* desarrollada por *Google*. Específicamente, se ha integrado el modelo *Face Landmarker*, una red neuronal altamente optimizada mediante *TensorFlow Lite*. La principal ventaja que proporciona el uso de este *framework* es que no se requiere una unidad de procesamiento gráfico dedicada (GPU) para garantizar un funcionamiento fluido, ya que el modelo está diseñado para ejecutarse en la CPU del equipo de manera ultrarrápida.

El modelo opera sobre una topología facial tridimensional conocida como *Face Mesh*, fundamentada en la identificación y el rastreo en tiempo real de 478 puntos clave o *landmarks* del rostro. El *software* de NEURO-FOCUS aísla matemáticamente los conjuntos (*clusters*) de puntos de dicha malla que corresponden a los contornos de los párpados y del iris. Mediante este procedimiento, el sistema calcula la posición del cursor e identifica los microparpadeos. En primer lugar, a través del cálculo del centroide geométrico de estos puntos oculares, se determina el vector de la mirada del usuario, trasladando proporcionalmente estas coordenadas a la resolución absoluta del monitor. En segundo lugar, el análisis de la distancia euclidiana entre los *landmarks* del párpado superior e inferior permite detectar con precisión si el ojo se encuentra cerrado, información crucial para estabilizar el cursor durante la ejecución de los clics.

4.3. Interfaz de usuario (GUI): CustomTkinter.

Para la creación de la interfaz gráfica de usuario (GUI) de NEURO-FOCUS, se seleccionó la librería *CustomTkinter*. Esta herramienta extiende las capacidades

del *framework* nativo *Tkinter*, proporcionando una capa estética más moderna y una gestión avanzada de los componentes visuales (*widgets*).

La elección del uso de *CustomTkinter* presente a la librería estándar se fundamenta en el conjunto de requerimientos tales como la usabilidad, el diseño adaptativo y las necesidades de accesibilidad específicas. El primer punto de esa elección se basa en que los componentes nativos de *Tkinter* presentan una estética obsoleta y rígida poco alineada con los estándares de diseño actuales en sistemas operativos como *Windows* 10 u 11, asemejándose más a *Windows* vista. *CustomTkinter* resuelve esta limitación, ya que proporciona *widgets* los cuales son completamente personalizables: esquinas redondeadas, degradados, animaciones y un renderizado de alta definición.

Un factor importante es que esta librería integra de forma nativa el soporte automático para el modo oscuro y claro, para un usuario con discapacidad motora que depende del parpadeo voluntario y la fijación de la mirada frente a una pantalla durante periodos de tiempo prolongados, la fatiga visual se convierte en un problema crítico. El uso del modo oscuro permite una reducción drástica de la luz azul emitida por la pantalla, por tanto, el uso del modo oscuro permite reducir el cansancio ocular, facilitando que los algoritmos de visión artificial sigan detectando la pupila con la máxima precisión sin que el lagrimeo del ojo afecte en el proceso.

A nivel de arquitectura, la integración de la GUI presentaba un desafío complejo en materia de concurrencia. El bucle de eventos principal (*event loop*) de la interfaz es altamente sensible, por lo que su ejecución debe realizarse en un hilo único de forma síncrona y bloqueante. Si el motor de visión artificial de MediaPipe o la escucha de datos a través del puerto serie de la diadema se ejecutarán sobre ese mismo hilo, se produciría la saturación del flujo principal. Como consecuencia, la interfaz gráfica quedaría suspendida de manera instantánea, provocando el bloqueo del sistema y la interrupción de la experiencia del usuario.

Para solucionar este conflicto, la arquitectura de la aplicación se diseñó de forma asíncrona. En esta estructura, la GUI actúa como el hilo principal (*main thread*) y se comunica con los hilos secundarios, encargados de la captura de vídeo y la telemetría de la diadema mediante variables de entorno seguro para hilos (*thread-safe*). Asimismo, se implementó el método `.after()` nativo de la librería, el cual permite programar actualizaciones periódicas de la interfaz en intervalos de pocos milisegundos. Esta estrategia facilita la actualización de la imagen de la

cámara, su máscara de puntos clave y el estado de conexión de la diadema, sin comprometer la fluidez ni la experiencia del usuario.

Para concluir el diseño del *layout* se ha estructurado de manera minimalista mediante una barra lateral fija de configuración y un área de trabajo limpia. Esta distribución estratégica garantiza que los controles esenciales (calibración, ajuste de sensibilidad del parpadeo e inicio del sistema) estén accesibles con el menor recorrido visual posible, optimizando el rendimiento del *eye-tracking* y reduciendo la carga cognitiva del usuario.

4.4. Hardware y biosensores: Diadema NeuroSky MindWave Mobile 2.

El componente de hardware seleccionado para la captura de señales bioeléctricas en NEURO-FOCUS es la diadema comercial NeuroSky *MindWave Mobile 2*[3]. Este dispositivo se clasifica dentro de las tecnologías BCI de tipo no invasivo y destaca en el mercado por su naturaleza de bajo coste, portabilidad y el uso de un biosensor seco (*dry sensor*), lo que elimina la necesidad de aplicar geles conductores en el cuero cabelludo del usuario.

A nivel de arquitectura física, el dispositivo consta de un brazo flexible que posiciona un único electrodo activo en la frente del usuario, concretamente en la ubicación Fp1, conforme al Sistema Internacional 10-20 estandarizado en electroencefalografía. Asimismo, incorpora un clip de referencia acoplado al lóbulo de la oreja con el propósito de mitigar el ruido eléctrico ambiental. El núcleo del hardware está integrado por el chip TGAM (*ThinkGear Amplifier*)[7], un módulo especializado que amplifica las micro señales bioeléctricas analógicas, que filtran las interferencias de la red eléctrica (50 Hz / 60 Hz) y digitaliza la información a una frecuencia de muestreo nativa de 512 Hz.

La transmisión de los paquetes de datos telemetrados hacia el ordenador se realiza de forma inalámbrica mediante un módulo Bluetooth integrado, estableciendo una comunicación serie asíncrona dedicada. Aunque el fabricante orienta originalmente este *hardware* a la monitorización de ondas cerebrales EEG (clasificadas en bandas Alfa, Beta, Theta y Gamma para medir estados cognitivos de atención y relajación), el software de NEURO-FOCUS aprovecha el canal de datos crudos (*raw data*) a 512 Hz.

A través de esta alta tasa de muestreo, el sistema es capaz de aislar los artefactos electromusculares (señales EMG) de gran amplitud generados por el músculo frontal al realizar un parpadeo voluntario o al fruncir el ceño. Al priorizar esta lectura electromuscular sobre la neuro eléctrica pura, el sistema evita los tiempos de cómputo y la ambigüedad de los algoritmos de clasificación mental, garantizando un disparador mecánico inmediato con una latencia de procesamiento ínfima, ya que si se usaran las señales neuro eléctricas el tiempo de cómputo para realizar un clic estaría cerca de los 3 segundos, causando mucha fatiga al usuario.

4.5. Librerías auxiliares: PyAutoGUI, PySerial y NumPy.

Para finalizar la integración de NEURO-FOCUS con el sistema operativo y el *hardware* externo han sido usadas dos librerías, en primer lugar, *PyAutoGUI*, quien tiene como función actuar simulando un puente entre los cálculos matemáticos de *MediaPipe* y el entorno de *Windows*, esta librería proporciona la capacidad de simular de forma nativa los movimientos del cursor y la ejecución de los clics, traduciendo los cálculos en acciones sobre la pantalla; En segundo lugar, se utiliza la librería *PySerial*, que es la que se encarga del establecimiento y la gestión de los puertos de conexión COM, gracias a esta, es posible que el programa pueda abrir un puerto de escucha continuo para recibir el flujo de datos en formato de *bytes*, que es enviado por la diadema mediante una conexión *Bluetooth*, permitiendo el procesamiento en tiempo real de las señales bioeléctricas.

5. Metodología y desarrollo del proyecto

5.1. Metodología de desarrollo ágil: Sprints y prototipado rápido.

Para el diseño y la construcción del software NEURO-FOCUS, se descartó desde el inicio el uso de metodologías tradicionales o secuenciales, tales como el modelo en cascada. Debido al carácter innovador del proyecto, el cual integra hardware biomédico, visión e inteligencia artificial, la investigación presentaba un alto grado de incertidumbre técnica. Por tanto, se requería un enfoque más flexible que permitiera la adaptación continua ante los imprevistos del desarrollo.

Factores como la variación en cuanto a la iluminación, la latencia de conexión *Bluetooth* o la precisión en la lectura de los micro parpadeos, hacían imposible que todos los requisitos fueran predefinidos de forma precisa y rígida desde el primer día de desarrollo.

Por este motivo, se adoptó el uso de una metodología de desarrollo ágil, inspirada en los principios del marco del trabajo *scrum*, pero con adaptaciones al trabajo individual. Esta implementación ha permitido flexibilizar el proceso dividiendo la carga de trabajo en ciclos cortos de desarrollo denominados *sprints*, con unas duraciones de entorno a una o dos semanas.

La gestión de las tareas dentro de cada *Sprint* se organizó mediante la filosofía *Kanban* (utilizando un tablero visual de estados: *To Do*, *In Progress*, *Testing*, *Done*). Esta organización garantizó que en cada ciclo se produjera un incremento funcional del *software*, por ejemplo, el primer ciclo se centró exclusivamente en lograr que la cámara detectase el rostro; el segundo, en traducir esos puntos a coordenadas; y el tercero, en captar la señal de la diadema, aislando la complejidad de cada subsistema.

Un pilar fundamental de esta metodología ha sido el prototipado rápido, en lugar de intentar programar la aplicación perfecta desde cero, se optó por construir Productos Mínimos Viables (MVP, por sus siglas en inglés). Mediante esta técnica, se programaron *scripts* aislados en Python (prototipos) para probar la viabilidad de MediaPipe y la lectura del puerto COM con *PySerial* de forma independiente. Una vez validado que el rendimiento computacional era óptimo y que los algoritmos matemáticos respondían adecuadamente en crudo, estos prototipos

se refactorizaron, optimizaron e integraron en la arquitectura final orientada a objetos y controlada por la interfaz gráfica de *CustomTkinter*.

Esta evaluación continua y temprana ha minimizado los riesgos de integración, permitiendo pivotar y modificar la lógica de código (Como la creación del algoritmo de "Freno de Mano") en cuanto se detectaron las primeras desviaciones durante las pruebas de usabilidad con la mirada.

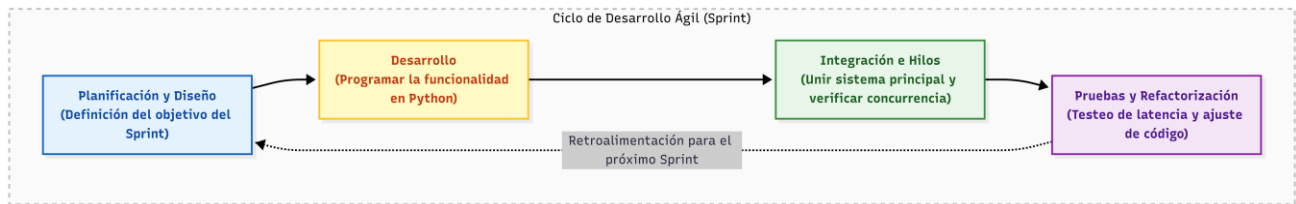


Ilustración 1. Ciclo iterativo del desarrollo aplicado en NEURO-FOCUS

5.2. Planificación temporal: Cronograma y Diagrama de Gantt.

La planificación temporal de NEURO-FOCUS se ha estructurado para garantizar el cumplimiento de los plazos de entrega establecidos para la recta final del mes de mayo, distribuyendo de forma eficiente las 200 horas de trabajo estimadas en la fase de presupuestación. Para el control y seguimiento de la evolución del proyecto, se ha diseñado un cronograma basado en un Diagrama de *Gantt*.

La temporalización se ha dividido en cuatro grandes bloques secuenciales:

Fase 1 (Investigación y Diseño): Dedicada al estudio de la viabilidad técnica de *MediaPipe* frente a otras alternativas de seguimiento ocular (*eye-tracking*), la selección de la diadema *NeuroSky* y el bocetado de la interfaz gráfica.

Fase 2 (Desarrollo del Core): El bloque más extenso, centrado en la programación en *Python*. Durante esta etapa se resolvieron los cuellos de botella del procesamiento de vídeo y se desarrolló el algoritmo matemático de estabilización.

Fase 3 (Pruebas y Depuración): Orientada a las pruebas de estrés del sistema, midiendo la latencia de los clics y ajustando la precisión del modelo predictivo.

Fase 4 (Documentación y Cierre): Reservada para las semanas previas a la entrega, englobando la redacción de la presente memoria técnica, el manual de usuario y la compilación del código fuente en su versión ejecutable.

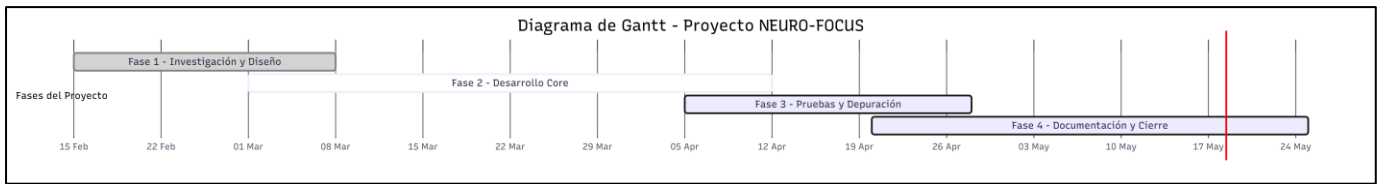


Ilustración 2: Diagrama de Gantt con la distribución temporal del proyecto NEURO-FOCUS.

5.3. Diseño de la Arquitectura: Sistema multihilo (Visión, Diadema y GUI).

Uno de los mayores desafíos técnicos en el desarrollo de NEURO-FOCUS radicó en la gestión de la concurrencia. El sistema operativo requiere que la interfaz gráfica de usuario se ejecute en un bucle principal (*main loop*) continuo para detectar las interacciones y renderizar los elementos visuales. En paralelo, el programa debe capturar de forma ininterrumpida fotogramas de vídeo a una tasa igual o superior a 30-60 FPS, procesarlos mediante redes neuronales y, simultáneamente, escuchar un puerto serie (COM) a una frecuencia de 512 Hz para recibir la telemetría de la diadema. Si estos procesos bloqueantes se ejecutaran secuencialmente en un único hilo, la interfaz gráfica perdería la capacidad de respuesta inmediatamente, provocando el bloqueo de la aplicación.

Para solventar este problema de arquitectura, se ha implementado un modelo de ejecución asíncrona en paralelo utilizando la librería nativa *threading* de Python. El software ha sido segmentado en tres hilos de ejecución principales, independientes y autónomos:

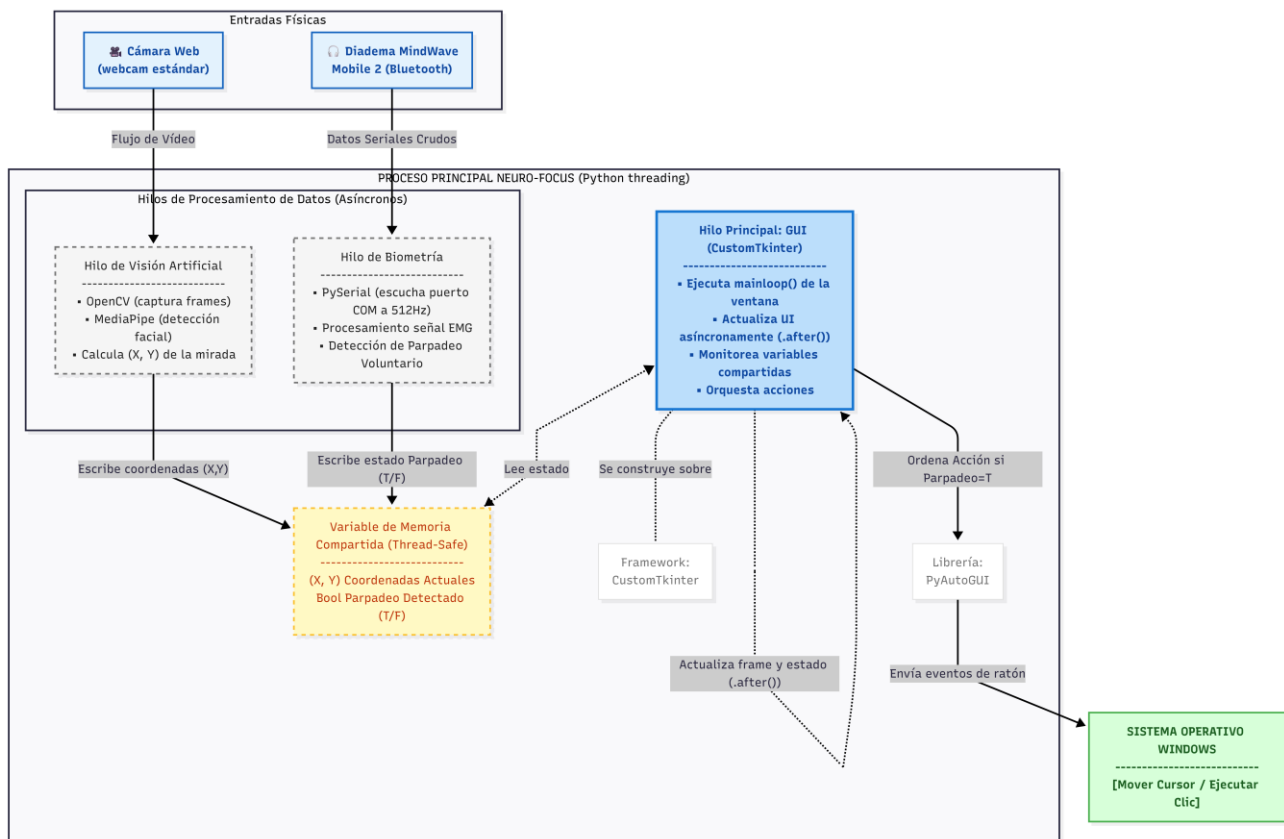


Ilustración 3. Diagrama de Arquitectura Multihilo Concurrente de NEURO-FOCUS

Tal y como se muestra en la ilustración el sistema ha sido segmentado en tres hilos de ejecución diferenciados.

Hilo Principal (GUI): Constituye el hilo base de la aplicación y se encarga exclusivamente de la gestión y el renderizado de la ventana de *CustomTkinter*. Este hilo no ejecuta cálculos computacionales pesados ni esperas bloqueantes. Para actualizar la imagen de la cámara web en la pantalla o modificar los indicadores de estado, se emplea el método *.after()*, el cual programa lecturas periódicas de los datos en intervalos de pocos milisegundos sin interrumpir la fluidez visual de la interfaz gráfica.

Hilo de Visión Artificial (Productor 1): Se encarga de abrir la conexión con *OpenCV*, leer los fotogramas en crudo y pasárselos al modelo de *MediaPipe*. Una vez que calcula el centroide de la mirada, almacena las coordenadas resultantes (X, Y) en variables de memoria compartidas y seguras (*thread-safe*).

Hilo de Biometría (Productor 2): Un hilo dedicado exclusivamente a mantener abierto el puerto COM mediante la librería *PySerial*, su única función es escuchar el flujo constante de bytes de la diadema. Cuando el algoritmo detecta un pico de señal electromuscular (EMG) correspondiente a un parpadeo, cambia el valor de una variable de estado booleana (*blink_detected* = True).

Finalmente, la integración de estos tres ecosistemas se produce a través de la librería *PyAutoGUI*[4]. Una función puente evalúa continuamente las variables compartidas: si el hilo de biometría indica un parpadeo, extrae las últimas coordenadas registradas por el hilo de visión y ordena al sistema operativo que ejecute el clic físico en esa posición exacta.

5.4. El Motor Visual:

5.4.1. Algoritmo de calibración dinámica de 9 puntos y extracción de marcadores faciales.

El motor visual de NEURO-FOCUS está impulsado por las redes neuronales de *MediaPipe Face Mesh*, una herramienta que mapea el rostro del usuario devolviendo una malla tridimensional compuesta por 478 marcadores faciales. Para lograr el control del cursor, el sistema discrimina la mayor parte de esta malla y aísla exclusivamente los puntos correspondientes a la mirada.

En concreto, el algoritmo extrae en tiempo real las coordenadas del marcador 468 (que representa el centro del iris derecho) y el marcador 473 (centro del iris izquierdo). Estos marcadores devuelven sus coordenadas en valores normalizados comprendidos entre 0.0 y 1.0. Sin embargo, dado que cada usuario tiene una fisionomía distinta, utiliza monitores de diferentes resoluciones y se sienta a una distancia variable de la *webcam*, resulta matemáticamente imposible hacer un mapeo directo entre el ojo y el cursor sin un paso previo de sincronización espacial. Para solventar esto, se ha implementado un algoritmo de calibración dinámica, en lugar de realizar una simple interpolación lineal, el sistema emplea una lógica de escalado basada en el centro geométrico del monitor, primero, obtiene una predicción cruda de las coordenadas en pantalla y le aplica unos valores de corrección de desvío (*offsets*), a continuación, calcula la

distancia de ese punto respecto al centro de la pantalla y la amplifica utilizando multiplicadores de sensibilidad personalizados (M_x Y M_y). La fórmula matemática que define este comportamiento en el eje horizontal es la siguiente:

$$X_{final} = C_x + (X_{crudo} + Offset_x - C_x) \times M_x$$

(Donde C_x representa el centro de la pantalla, X_{crudo} la predicción inicial y M_x el multiplicador de sensibilidad).

A nivel de desarrollo, esta lógica matemática ha sido integrada en el hilo de visión artificial de *Python* mediante una función de mapeo dedicada. A esta función se le ha añadido un delimitador de bordes (*clamping*) para asegurar que la conversión se realice sin desbordamientos y en fracciones de milisegundo para cada fotograma capturado:

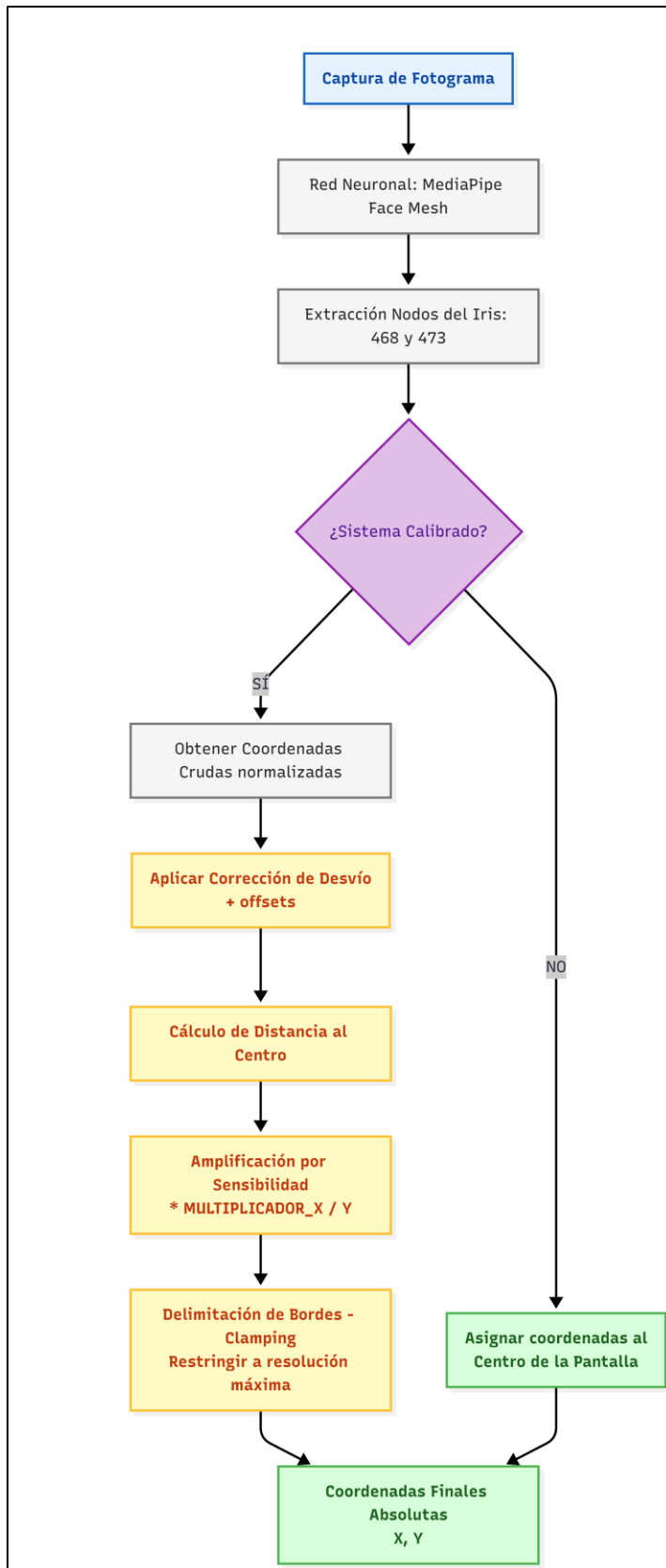


Ilustración 4: Diagrama de flujo del algoritmo de calibración y mapeo del espacio

```
def map_to_screen(self, eye_x, eye_y):
    #si no existe una calibracion va al centro de la pantalla
    if not self.is_calibrated: return screen_w / 2, screen_h / 2
    #obtencion de las cordenas crudas
    sx, sy = self._predict_raw(eye_x, eye_y)
    #aplicacion de los valores de corrección llamados offsets
    sx += self.offset_x
    sy += self.offset_y
    #cálculo del centro de la pantalla
    cx, cy = screen_w / 2, screen_h / 2
    #escalado de la sensibilidad del centro al borde
    final_x = cx + (sx - cx) * MULTIPLICADOR_X
    final_y = cy + (sy - cy) * MULTIPLICADOR_Y
    #delimitado de bordes
    return max(0, min(screen_w - 1, final_x)), max(0, min(screen_h - 1, final_y))
```

Este enfoque garantiza que el rango de movimiento limitado del globo ocular se amplifique de forma dinámica para cubrir la totalidad del área de la pantalla. De este modo, se proporciona una navegación precisa e independiente de las características del *hardware* utilizado por el usuario.

5.4.2. Implementación del Filtro de Kalman para la estabilización del cursor.

A pesar de que la malla facial de *MediaPipe* presenta una alta precisión, la captura de video con *webcam* está sujeta a imperfecciones presentes en el entorno, tales como variaciones en la iluminación, ruido en el sensor de la lente, o provocados por el usuario, como son los micromovimientos involuntarios del ojo. Si esos datos fueran enviados de forma cruda directamente tras su extracción, la calibración se enviaría de forma instantánea al sistema operativo, por tanto, el curso sufriría de un efecto de “temblor” continuado, llegando a producir que algunas funciones fueran casi imposibles de realizar.

Para solucionar este problema y suavizar la trayectoria del cursor sin introducir una latencia excesiva, se ha integrado en el Motor Visual un Filtro de Kalman[1]. Este algoritmo matemático, ampliamente utilizado en sistemas de navegación y control, no se limita a promediar los últimos movimientos, sino que utiliza un modelo predictivo.

En cada fotograma el filtro de Kalman realiza dos fases:

Fase1 predicción. El modelo calcula donde debería de estar el cursor en ese instante basado en factores como velocidad y su trayectoria previa.

Fase 2 actualización/corrección. El modelo toma la nueva medida cruda que ha sido capturada por *MediaPipe* y la compara con su predicción, aplicando una ganancia para decidir cuál de los dos valores presenta una mayor fiabilidad.

La base matemática de la fase de actualización es definida por esta ecuación.

$$X_K = X_{K-1} + K \times (Z_k - X_{k-1})$$

(Donde X_K es la posición final suavizada, X_{K-1} es la posición predicha, Z_K es la medición cruda actual del ojo, y K representa la Ganancia de Kalman, que pondera el nivel de ruido del sistema). A nivel de código, esta compleja lógica matemática se ha abstraído e implementado de forma eficiente aprovechando el módulo nativo de la librería *OpenCV* (*cv2.KalmanFilter*). Mientras que la configuración de matrices del filtro se inicializa al arrancar el programa, la estabilización en tiempo real se ejecuta con un bloque muy conciso justo antes de mover el ratón:

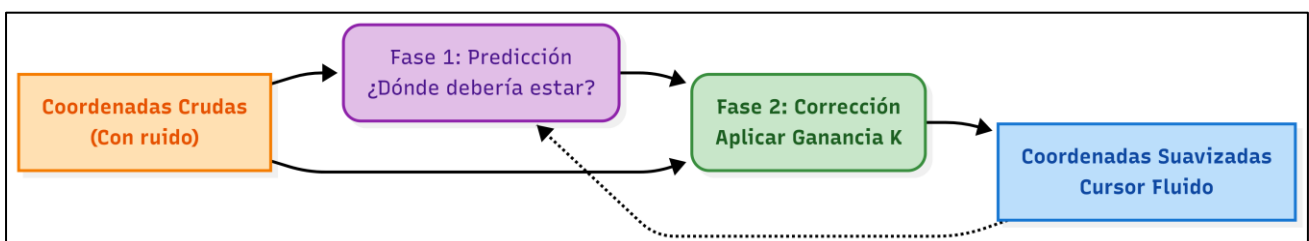


Ilustración 5. Funcionamiento del filtro de Kalman en sus fases.

```

calibrator = AdvancedCalibrator()
#inicio del filtro 4 variables dinamicas y 2 de medición
kalman = cv2.KalmanFilter(4, 2)
#matriz de medición(H) relaciona el estado interno con la medición
kalman.measurementMatrix = np.array([[1, 0, 0, 0], [0, 1, 0, 0]],

```

```
dtype=np.float32)
# matriz de transicion (f) es el modelo fisico funcionamiento =
# posicion actual + velocidad
kalman.transitionMatrix = np.array([[1, 0, 1, 0], [0, 1, 0, 1], [0,
0, 1, 0], [0, 0, 0, 1]], dtype=np.float32)
#matriz de covarianza / ruido del proceso (Q) es la confianza que
#tenemos en el modelo fisico, a mayor valor mas lento se adapta a
#cambios rapidos
kalman.processNoiseCov = np.eye(4, dtype=np.float32) * 0.01
#matriz de covarianza para el ruido en la medicion (R) encargado
#del ajuste de tembores
kalman.measurementNoiseCov = np.eye(2, dtype=np.float32) * 15.0
```

Gracias a la integración de este filtro predictivo, el cursor final en la pantalla del usuario adquiere una fluidez absoluta, eliminando el ruido del sensor de la cámara, pero manteniendo una respuesta casi instantánea a los movimientos intencionados de la cabeza o la mirada.

5.5. El Motor de Clic:

5.5.1. Procesamiento de señales crudas de la diadema.

El motor encargado del clic en NEURO-FOCUS es completamente dependiente de la captura y análisis de la actividad electromuscular, (EMG)[7] que es generada por el usuario mediante la realización de un parpadeo voluntario. Para que eso sea posible, se establece una conexión asíncrona con la diadema a través del protocolo de comunicación *Bluetooth junto con la emulación de un puerto de comunicaciones serie(COM) tradicional*.

A nivel de arquitectura de *hardware*, se ha optado por un modelo de configuración explícita para la asignación de dicho puerto, en lugar de implementar un escaneo automatizado en segundo plano (que podría generar conflictos de acceso o latencia con otros periféricos *Bluetooth* del sistema operativo), el *software* delega la selección del puerto COM al usuario final, esta decisión de diseño garantiza una conexión más estable y predecible, otorgando al usuario el control total sobre su entorno físico. El procedimiento para emparejar el dispositivo y forzar la asignación del puerto en el sistema operativo *Windows* se encuentra exhaustivamente documentado en el Manual de Usuario adjunto a este proyecto.

En cuanto a la toma de datos, el hilo de la biometría (productor 2) utiliza la librería llamada *PySerial*[5] para realizar una apertura de ese puerto con una configuración de velocidad de transmisión estándar en 57600 baudios, en el que la diadema emite un flujo de datos de manera constante e ininterrumpida de *bytes* en una frecuencia de 512Hz.

Con el propósito de procesar este volumen de información en tiempo real, minimizar la latencia y evitar el bloqueo del sistema, el algoritmo implementa un analizador sintáctico (*parser*). Este componente se encarga de segmentar y validar las tramas de datos recibidas antes de su integración en el flujo principal.

El analizador examina de manera continua el puerto de entrada con el fin de identificar una secuencia específica de sincronización, constituida por los bytes de cabecera 0xAA. Una vez detectada esta estructura, el sistema extrae el paquete de información o carga útil (*payload*). A partir de dicho paquete, se descartan de inmediato los valores irrelevantes para los objetivos de este proyecto (tales como los niveles de atención o meditación) y se aísla la amplitud de la señal electromuscular asociada a los parpadeos del usuario.

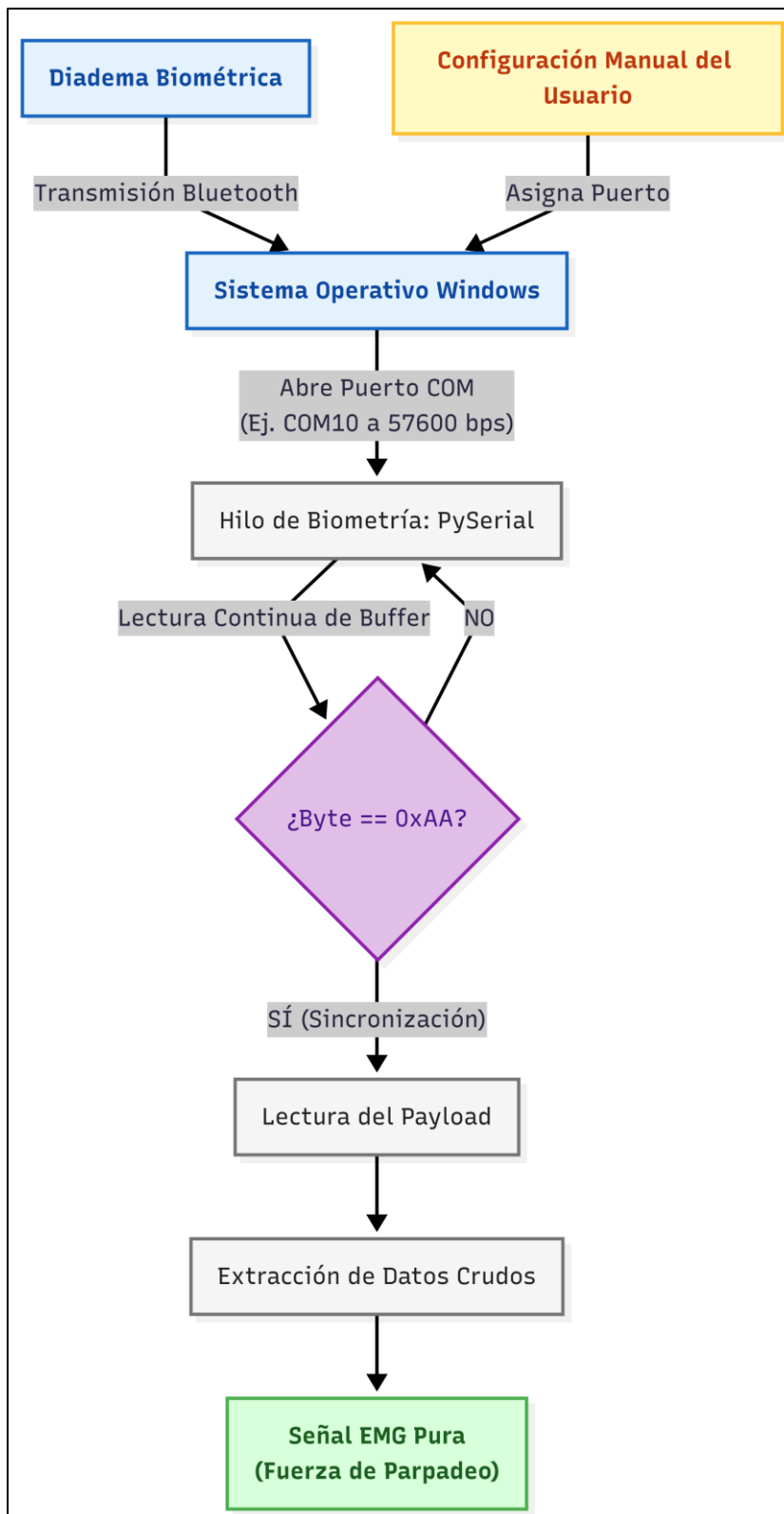


Ilustración 6. Diagrama de flujo de la toma de datos y su decodificado mediante *PySerial*.

A nivel de código, esta recepción y sincronización se gestiona de la siguiente manera dentro del hilo dedicado.

```
while self.running and self.diadema.is_open:
    try:
        #comprobar si hay datos de entrada en el puerto serie
        if self.diadema.in_waiting > 0:
            #Buscar el inicio del paquete (0xAA 0xAA)
            if self.diadema.read() == b'\xaa' and
self.diadema.read() == b'\xaa':
                payload_len = self.diadema.read()
                if not payload_len: continue
                p_len = ord(payload_len)
                if p_len > 170: continue #borrado de corruptos
                # leer paquete de datos completo y el checksum (1
byte) encargado de la validacion
                payload = self.diadema.read(p_len)
                self.diadema.read() # checksum

                i = 0
                #analisis de cargaa en busca de los codigos de
calidad de señal y parpadeo
                while i < p_len:
                    code = payload[i]
                    if code == 0x02: # Calidad de señal
                        sq = payload[i + 1]

                        if sq == 0: # si la señal es cero el
contacto sensor usuario es perfecto
                            tiempo_señal_perfecta = time.time()
```

Esta arquitectura garantiza que los bytes no se pierdan durante la transmisión, dando una base sólida con baja latencia para el punto 5.5.2 en el sistema de gatillo fácil.

5.5.2. Lógica del "Gatillo Fácil" basada en calidad de señal.

Durante la fase de pruebas de usabilidad con la diadema *NeuroSky*[3], se detectó un problema técnico crítico: depender exclusivamente del código oficial de parpadeo (0x16) obligaba al usuario a guiñar el ojo con demasiada fuerza para que el sistema registrase la acción, lo que provocaba una rápida fatiga ocular y frustración.

Para solucionar este cuello de botella y maximizar la accesibilidad, se desarrolló el algoritmo denominado "Gatillo Fácil", este sistema no se espera a recibir el paquete oficial de parpadeo, sino que monitoriza continuamente la variable en crudo de la calidad de la señal. El funcionamiento se basa en un principio físico: cuando el sensor tiene un contacto perfecto con la piel, el valor es 0. Al realizar un parpadeo voluntario (EMG), se produce un micro movimiento instantáneo que altera la resistencia de la piel y eleva el ruido, haciendo que $sq > 0$ momentáneamente.

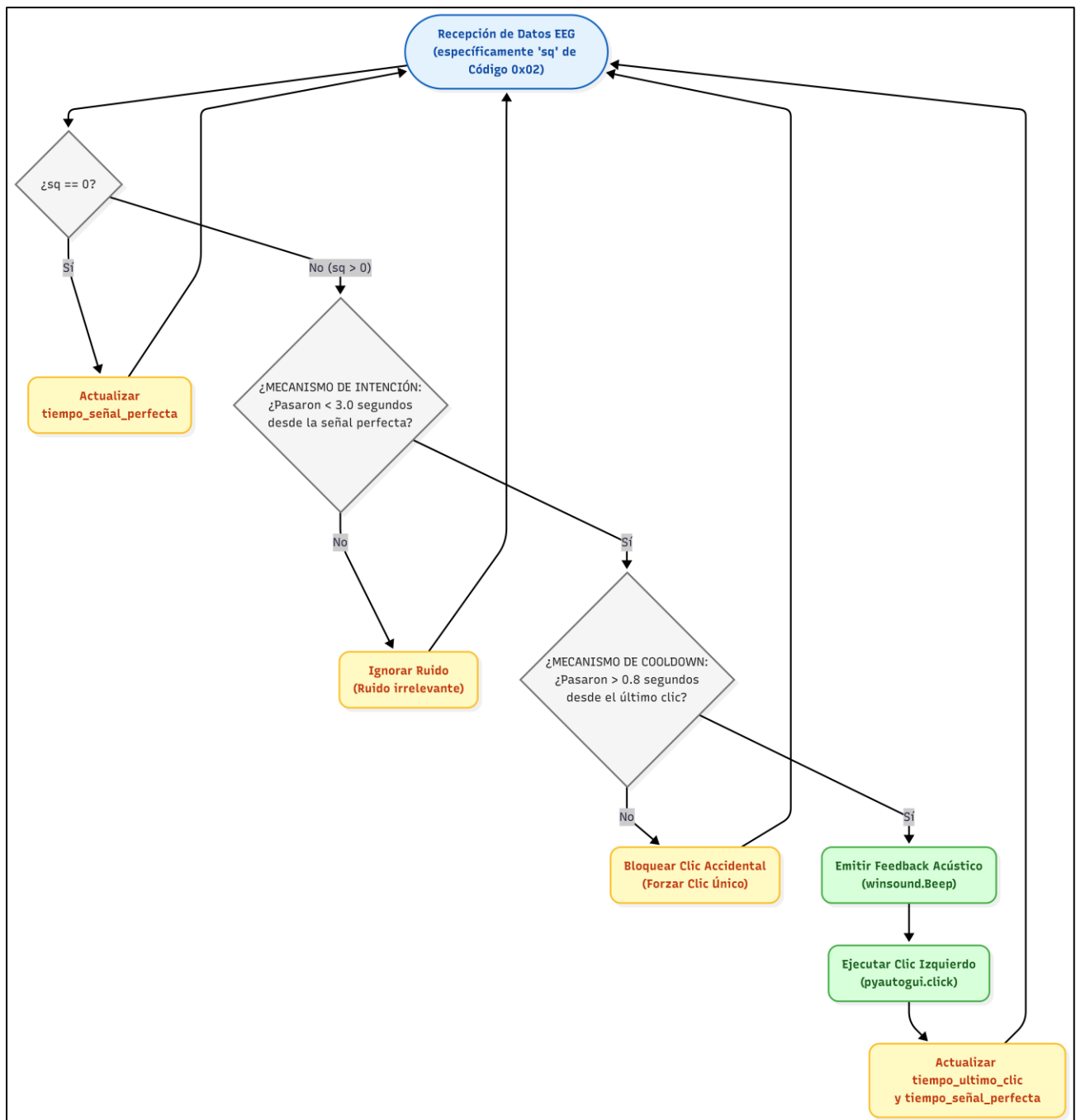


Ilustración 7: Diagrama de funcionamiento del algoritmo "Gatillo Fácil" y gestión del clic.

Tal y como se muestra en el diagrama, el algoritmo gestiona la acción mediante las siguientes directrices lógicas y de usabilidad:

Detección de Intención: Si el programa detecta una alteración de ruido ($sq > 0$) a menos de 3.0 segundos de haber registrado una "señal perfecta (se produce cuando la variable sq es igual a 0)", interpreta que el usuario ha querido parpadear sin necesidad de forzar el gesto facial.

Sinergia con el sistema operativo, clic único y supresión del clic derecho: Forzar al usuario a realizar un doble parpadeo rápido para emular el doble clic tradicional resultaba inviable por el cansancio físico que conlleva, además de ocasionar fallos ante parpadeos reflejos. Por este mismo motivo de diseño y reducción de carga cognitiva, se ha prescindido por completo del clic secundario (clic derecho) y del menú contextual. Para compensar esta ausencia, el proyecto exige la activación de la configuración nativa de *Windows* "Un solo clic para abrir elemento". Al unificar toda la navegación del sistema operativo en una única acción de interacción sencilla, se elimina la necesidad de realizar gestos faciales complejos o combinados.

Mecanismo de Control (*Cooldown*): Al no existir la necesidad del doble clic, se ha implementado un mecanismo de enfriamiento de 0.8 segundos mediante la librería *time*. Si no ha transcurrido ese tiempo desde la última acción, el sistema ignora cualquier nuevo parpadeo, garantizando así una pulsación limpia y evitando ametralladoras de clics involuntarios.

Retroalimentación Acústica: Al cumplir las condiciones, se ejecuta el clic mediante *pyautogui.click()* y, simultáneamente, se emite un pitido grave (1000 Hz) mediante *winsound*. Esta confirmación sonora es crucial desde el punto de vista de la experiencia de usuario (UX) para asegurar que la orden ha sido procesada correctamente, supliendo la falta de sensación táctil de un ratón tradicional.

En términos de código la implementación de esa lógica se realizó de una forma muy concreta tras el proceso de análisis de la señal.

```
# HACK SÚPER SENSIBLE
if sq > 0 and (time.time() - tiempo_señal_perfecta < 3.0):
    #gestion del tiempo entre clics
    if time.time() - tiempo_ultimo_clic > 0.8:
        #pitido de confirmacion
        try:
```

```
import winsound
winsound.Beep(1000, 150)
except:
    pass
#ejecucion del click
pyautogui.click()
tiempo_ultimo_clic = time.time()
tiempo_señal_perfecta = 0 #reinicio contador
```

5.6. Ingeniería de Usabilidad (UX):

5.6.1. Sistema de Freno de Mano inteligente.

En la navegación mediante el uso de un sistema de *eye-tracking* existe en tipo de fatiga llamado fatiga por control, esto presenta un gran reto, ya que el usuario siente presión al saber que cada movimiento que realiza está moviendo el cursor, lo que produce molestias durante la lectura o descanso visual. Con el propósito de mitigar este problema sin necesidad de pausar o detener la aplicación por completo, se diseñó e implementó un módulo de suspensión temporal del rastreo. Este sistema actúa como un freno de mano inteligente.

Este mecanismo permite al usuario congelar la posición del cursor en un punto específico de la pantalla. Mientras tanto, el hilo de visión artificial continúa su ejecución en segundo plano, procesando los fotogramas y manteniendo la detección del iris, sin embargo, la instrucción de movimiento enviada al sistema operativo queda completamente suprimida. Esta estrategia permite realizar lecturas prolongadas o descansos visuales sin riesgo de desplazamientos accidentales del puntero. La función se activa de manera orgánica mediante el entornado o cierre leve de los ojos del usuario

La arquitectura del código administra esta funcionalidad a través de la variable de bandera booleana *cursor_frozen* y un temporizador dedicado. El sistema implementa una lógica de conmutación basada en dos disparadores cinemáticos principales derivados del análisis facial: el estado de pausa se induce mediante el entornado sostenido de los ojos, mientras que la transición de retorno al estado activo se desencadena bien de forma automatizada ante una apertura ocular exagerada (*eye widening*), o bien de forma manual mediante una interacción física con los componentes de la GUI.

```
# --- LÓGICA DE CONGELACIÓN con cálculo de la distancia euclidiana
entre parpados
eye_openness = math.hypot(lm[159].x - lm[145].x, lm[159].y -
lm[145].y)
EYE_HISTORY.append(eye_openness)
if len(EYE_HISTORY) > 8: EYE_HISTORY.pop(0)
avg_openness = float(np.mean(EYE_HISTORY))
#activacion por gestos oculares
if calibrator.is_calibrated:
    #si el ojo se entorna <threshold_close> o se abre mucho >
    <threshold> se activa
    if blink_state == "NORMAL" and (
        avg_openness < BLINK_THRESHOLD_CLOSE or avg_openness >
        BLINK_WIDEN_THRESHOLD):
        blink_state = "DEFORMADO"
        cursor_frozen = True
        #temporizador de congelacion
        tiempo_descongelacion = time.time() + 1.2
```

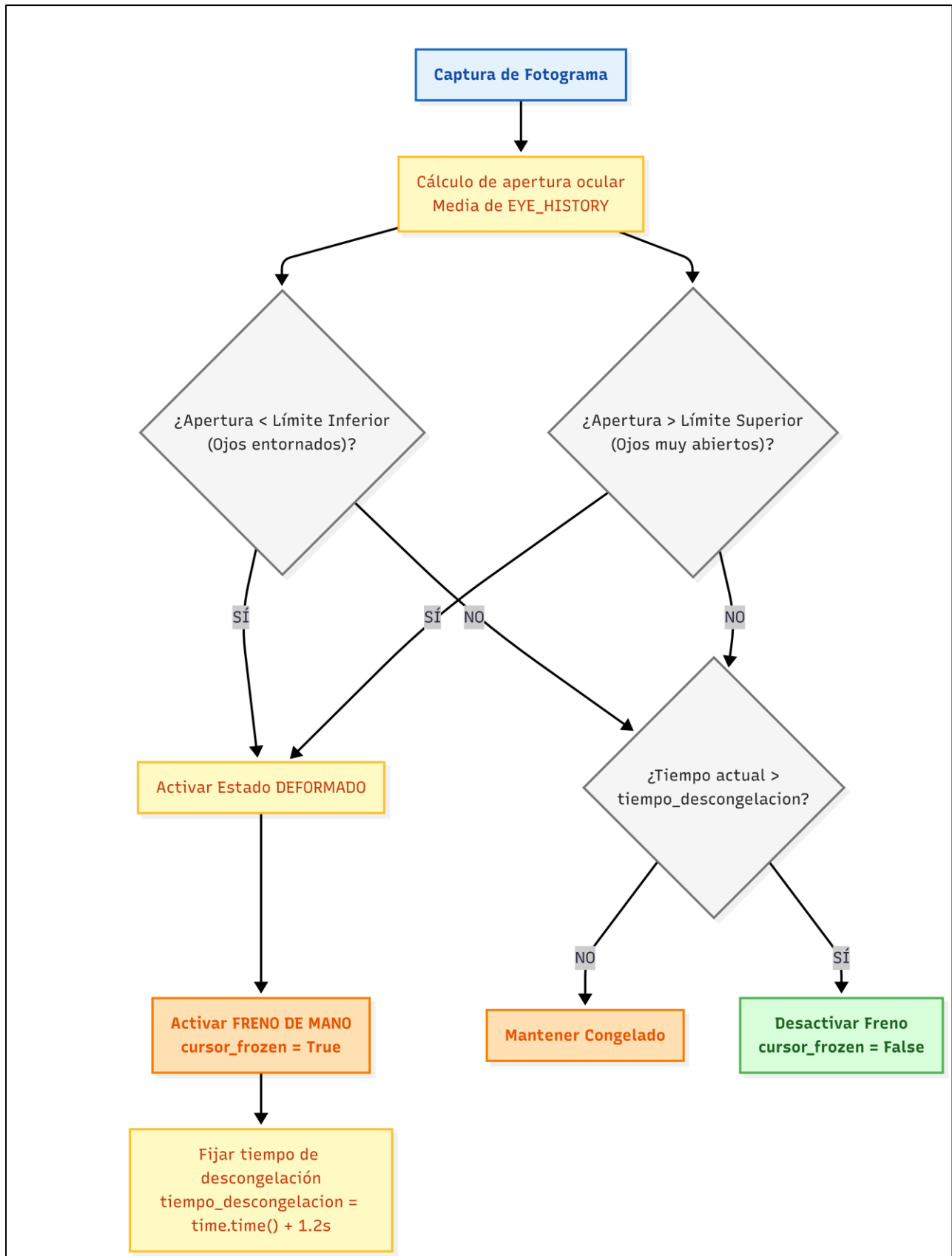


Ilustración 8. Diagrama del funcionamiento del freno de mano.

5.6.2. Mecanismo de "Rollback Temporal" (Máquina del tiempo).

La deriva posicional previa a la interacción constituye uno de los mayores obstáculos arquitectónicos en los sistemas de seguimiento ocular. Al parpadear voluntariamente para disparar una acción, la contracción de los músculos faciales deforma la geometría periocular, alterando temporalmente la malla tridimensional de *MediaPipe*. Como consecuencia de este desfase, el sistema calcula un cambio de trayectoria inexistente en la mirada, traduciéndose en un desplazamiento involuntario del cursor hacia la zona inferior milisegundos antes de consolidar el clic. La ejecución del evento en coordenadas erróneas comprometería gravemente la viabilidad operativa de la aplicación, volviendo el sistema inestable para el usuario final.

Para eliminar ese margen de error se ha diseñado un algoritmo de compensación espacial que ha sido nombrado mecanismo de *rollback* temporal, su funcionamiento se basa en que el sistema mantiene en memoria un buffer dinámico llamado (POSITION_HISTORY) que almacena de forma continua las coordenadas exactas del cursor correspondientes a los últimos fotogramas procesados, el tiempo es de unos 15 milisegundos, por lo que si ejecutas el clic en el segundo 1, el programa usará la posición previa a esa en 15 *frames*, lo que equivale a 0.5 segundos. De este modo, si el parpadeo se consolida en un instante concreto, el programa desecha las coordenadas deformadas por el movimiento muscular y rescata la posición segura guardada frames atrás en el búfer.

Cuando el evaluador de apertura ocular detecta que el usuario está comenzando a cerrar el ojo y entra en estado "DEFORMADO", el algoritmo interviene de forma inmediata. En lugar de permitir que el cursor acompañe a la deformación facial, el sistema extrae la coordenada más antigua almacenada en el búfer (que representa la posición "segura" de la mirada hace aproximadamente 0.5 segundos, antes de que el músculo comenzara a moverse) y teletransporta el ratón de vuelta a esa ubicación exacta.

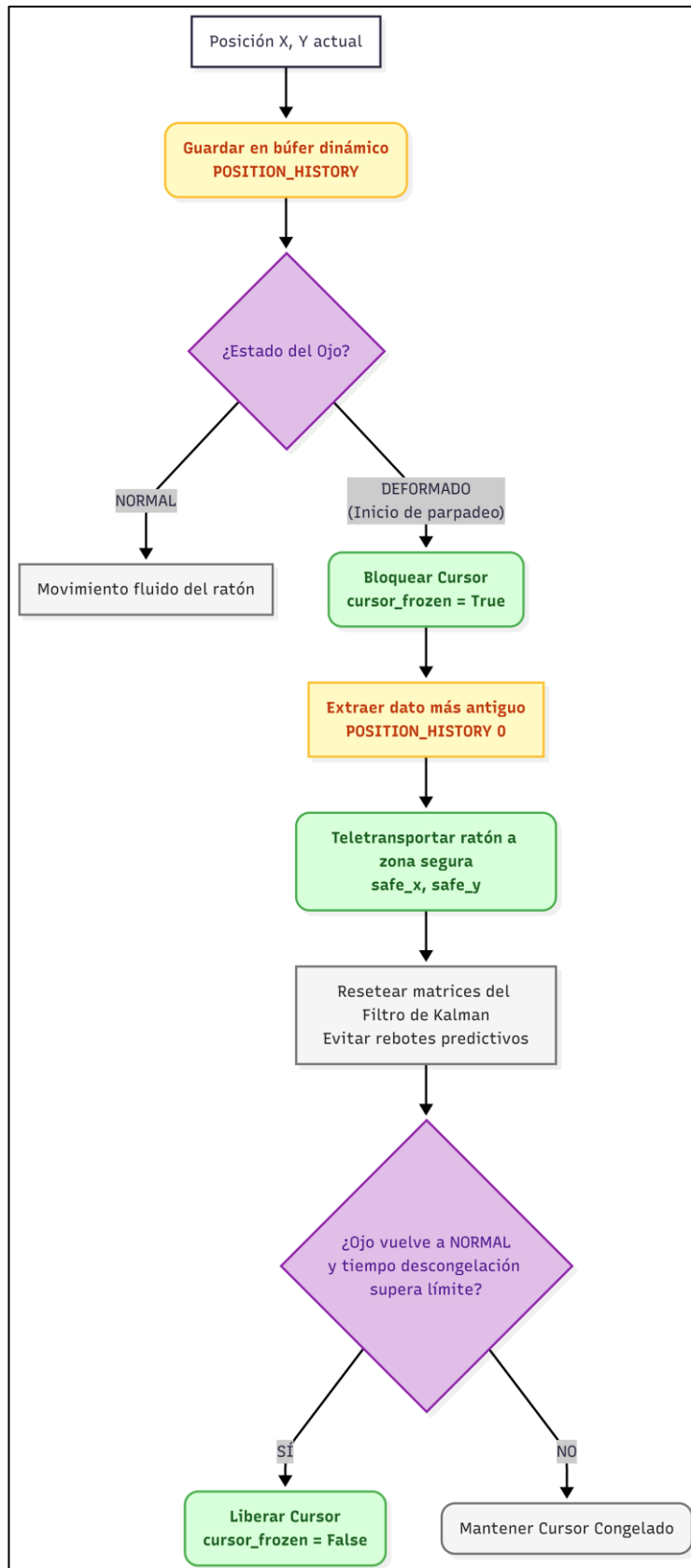


Ilustración 9. Diagrama de flujo del mecanismo de *rollback* o compensación.

A nivel de código, para ejecutar esta máquina del tiempo es necesario que las matrices predictivas del filtro de Kalman sean estabilizadas, ya que, si eso no sucediera, las variables de estado del filtro no serían reiniciadas con la posición segura para el clic y el modelo matemático entendería el desplazamiento como un error de medición del sensor o un movimiento involuntario, produciendo un rebote indeseado.

```
# --- LA MÁQUINA DEL TIEMPO (ROLLBACK) ---
# Si tenemos historial, recuperamos la posición de hace ~15
frames (0.5 segundos)
# Esto deshace la desviación causada por la ceja bajando antes
de hacer clic.
if len(POSITION_HISTORY) >= 5:
    safe_x, safe_y = POSITION_HISTORY[0] # Cogemos el dato más
antiguo
    pyautogui.moveTo(safe_x, safe_y) # Teletransporte del
ratón
    prev_mouse_x, prev_mouse_y = safe_x, safe_y
    # Estabilizamos el predictor de Kalman para que no pegue
tirones al despertar
    medida = np.array([[safe_x], [safe_y]], dtype=np.float32)
    kalman.correct(medida)
    kalman.statePost = np.array([[safe_x], [safe_y], [0], [0]],
dtype=np.float32)
    kalman.statePre = kalman.statePost.copy()
#Gestion de estados del parpadeo encargado de liberar el cursor
elif blink_state == "DEFORMADO" and (BLINK_THRESHOLD_CLOSE <
avg_openness < BLINK_WIDEN_THRESHOLD):
    blink_state = "NORMAL"

if blink_state == "NORMAL" and time.time() > tiempo_descongelacion:
    cursor_frozen = False
else:
    cursor_frozen = True
```

5.7. Integración de herramientas: Teclado virtual y dictado por voz.

El propósito básico y fundamental de la aplicación NEURO-FOCUS es proporcionar una autonomía total a usuarios que presentan una limitación de movilidad severa. Los apartados anteriores de esta memoria se han centrado en detallar la resolución técnica del control del cursor y la ejecución del clic, llegando a hitos como la sustitución con éxito de funciones que requieren de un ratón físico, hecho que sigue sin ser suficiente para que un sistema operativo hoy en día sea plenamente usable, por ello, es necesario garantizar una vía de entrada de texto eficiente, ya que el usuario necesita poder redactar correos electrónicos, utilizar el navegador o simplemente buscar una aplicación en el sistema de manera autónoma.

Durante la fase de diseño arquitectónico del proyecto, se evaluó la posibilidad de programar un teclado virtual propio integrado directamente dentro de la interfaz gráfica de la aplicación mediante la librería *CustomTkinter*. No obstante, desde el punto de vista de la ingeniería de *software* y la usabilidad, se concluyó que esta aproximación era ineficiente. Desarrollar un teclado interno supondría "reinventar la rueda", consumiendo recursos valiosos de la CPU, sobrecargando el hilo principal de la aplicación y generando posibles conflictos de superposición de ventanas (*z-index*) con otros programas del escritorio.

Por este motivo, se optó por una estrategia de arquitectura modular en la que se delegó la introducción y gestión de texto a las herramientas de accesibilidad nativas del sistema operativo *Windows*. Esta aproximación permite aprovechar un entorno robusto y optimizado, reduciendo la sobrecarga de procesamiento de la aplicación. El motor visual de NEURO-FOCUS y el funcionamiento del clic fue diseñado para ser completamente transparente y compatible con el teclado en pantalla de *Windows*, (OSK) al desplazar la mirada, el usuario puede navegar por las teclas virtuales con fluidez y, gracias al mecanismo de "Gatillo Fácil" accionado por la diadema biométrica, pulsar cada letra con total precisión. Además, al aprovechar las tecnologías de predicción de palabras que incluyen estos teclados nativos, la velocidad de escritura aumenta considerablemente, mitigando la fatiga ocular.

A pesar de esta optimización, la escritura de un texto de una longitud moderada mediante el seguimiento ocular resulta un proceso muy lento y agotador, para

dar solución a este cuello de botella y ofrecer algo realmente accesible, NEURO-FOCUS se apoya también en el dictado por voz.

El flujo de trabajo ideal permite al usuario utilizar la mirada y el parpadeo para navegar por la interfaz, abrir un procesador de textos, hacer clic en el campo de escritura y, posteriormente, activar el motor de reconocimiento de voz integrado en *Windows* (accesible mediante el botón de micrófono en el propio teclado virtual). Esta sinergia permite utilizar la visión artificial para el posicionamiento espacial (donde es extremadamente rápida y eficiente) y la voz para la ingesta masiva de caracteres (donde el dictado acústico es imbatible).

En resumen, la decisión de no incrustar herramientas de texto propietario responde a un análisis de eficiencia, estabilidad y modularidad, ya que al actuar como un simbiote de las funciones existentes ya en el sistema operativo, la aplicación se mantiene lo más ligera posible, enfocando todos los recursos en el procesamiento de las señales de la diadema y los algoritmos de visión artificial. La integración de esto culmina en una solución de accesibilidad integral y muy cómoda para el usuario.

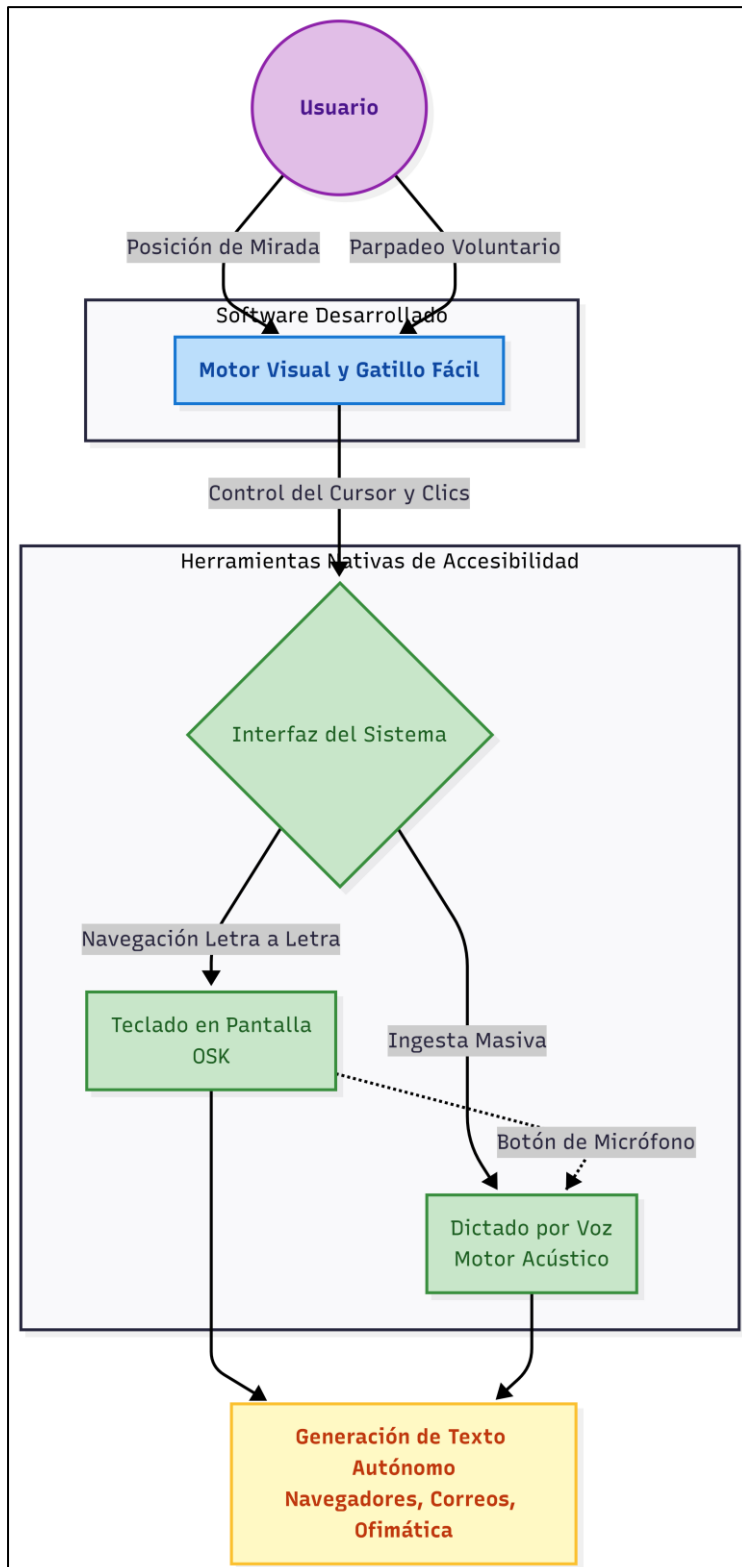


Ilustración 10. Diagrama de integración simbiótica entre NEURO-FOCUS y las herramientas de accesibilidad de Windows.

5.8. Despliegue: Generación y empaquetado del ejecutable (.exe).

El objetivo final en el ciclo de vida del desarrollo de NEURO-FOCUS es garantizar una distribución sencilla, accesible y eficiente para el usuario final. Dado que la aplicación está destinada a personas con discapacidades motoras severas, resulta inviable exigir la instalación previa del intérprete de *Python*, la configuración de entornos virtuales o la gestión de dependencias complejas a través de la consola de comandos. Por tanto, el proyecto requiere una fase de empaquetado que unifique todo el entorno de ejecución en un único archivo ejecutable de *Windows* (.exe) totalmente autónomo (*standalone*).

Para llevar a cabo este proceso de despliegue, se ha seleccionado la herramienta PyInstaller. Esta utilidad analiza el script principal de la aplicación, rastrea de forma recursiva todas las librerías importadas (como *OpenCV*, *MediaPipe*, *PySerial* y *CustomTkinter*) y junta los binarios y dependencias necesarias dentro de una estructura distribuible.

Sin embargo, el empaquetado de NEURO-FOCUS presentó dos desafíos técnicos críticos derivados de la gestión de recursos dinámicos. El primero de ellos se debió al elevado peso del modelo predictivo de Inteligencia Artificial, almacenado en el archivo `face_landmarker.task`. Para solventar esta problemática, se implementó un sistema de descarga de recursos bajo demanda apoyado en la propiedad `sys.frozen`. Mediante esta lógica, el código determina si la aplicación se está ejecutando como un binario empaquetado y verifica la existencia del modelo en la ruta local del sistema. En caso de no ser localizado (escenario habitual durante el primer inicio), un hilo secundario gestiona de forma automatizada su descarga desde los servidores de *Google*, evitando así el bloqueo de la interfaz gráfica de usuario (GUI).

El segundo desafío consistió en la omisión de las dependencias ocultas de C++. Dado que *MediaPipe* requiere bibliotecas dinámicas compartidas (como sus componentes .dll nativos), el motor de empaquetado de *PyInstaller* no logra identificarlas de manera automática. Esta limitación requirió una intervención manual en el script de configuración para forzar la inclusión de dichos binarios en el árbol del paquete.

Para solucionar estos conflictos y generar el producto final, el empaquetado definitivo se realiza a través de la interfaz de comandos utilizando la siguiente instrucción configurada con *flags* específicos de producción:

pyinstaller --noconfirm --onedir --noconsole --name "NEURO-FOCUS" --icon=icono.ico --add-data "icono.ico;." --collect-all "mediapipe" .\FINAL_NEURO.py

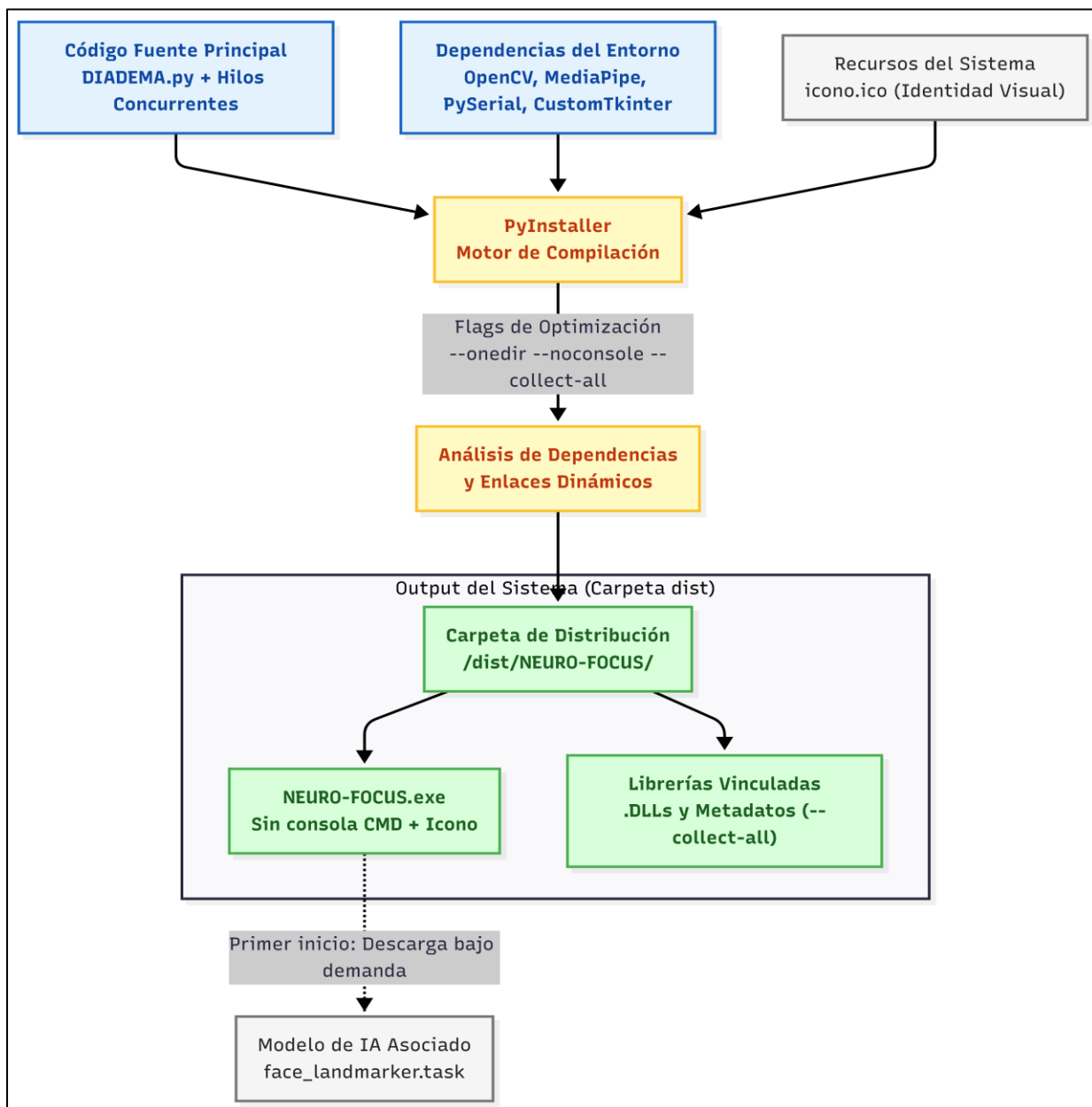


Ilustración 11: Diagrama de flujo del proceso de compilación y despliegue con PyInstaller.

El empaquetado definitivo se realiza a través de la interfaz de comandos utilizando la siguiente instrucción configurada con flags específicos de producción:

```
pyinstaller --noconfirm --onedir --windowed --name "NEURO-FOCUS" --clean  
main.py
```

La justificación técnica del uso de estos parámetros de empaquetado se detalla a continuación:

-  **--onedir:** Genera una carpeta de distribución que contiene el ejecutable principal junto con las librerías dinámicas (.dll). Se ha elegido esta opción en lugar de **--onefile** debido a que el tamaño de los módulos de inteligencia artificial de *MediaPipe* y *OpenCV* ralentizaría drásticamente el tiempo de arranque de la aplicación si tuviera que descomprimirse por completo en la memoria caché temporal cada vez que se inicia.
-  **--noconsole:** Suprime la apertura de la consola de comandos de *Windows* (cmd.exe) al arrancar el programa. Al tratarse de una aplicación orientada a la accesibilidad con entorno gráfico propio, mostrar la consola cruda penalizaría la estética y la experiencia de usuario (UX).
-  **--icon y --add-data:** Incrustan la identidad visual del proyecto, asignando el logotipo diseñado específicamente para NEURO-FOCUS tanto al ejecutable como a la barra de tareas.
-  **--collect-all "mediapipe":** Fuerza al motor de compilación a recolectar absolutamente todos los subdirectorios, metadatos y enlaces dinámicos ocultos de la librería de *MediaPipe*, evitando errores de ejecución de tipo `ModuleNotFoundError` o pérdida de archivos .dll.

Una vez finalizado el proceso, el instalador genera la carpeta autónoma lista para ser distribuida mediante un asistente de instalación estándar, logrando que el usuario final pueda disfrutar de una accesibilidad completa con un simple doble clic.

6. Medios y cálculo de costes

6.1. Recursos materiales y hardware.

Para el desarrollo y la puesta en producción del proyecto de NEURO-FOCUS, la prioridad principal ha sido el uso de un equipamiento accesible al público, evitando el coste de adquirir servidores o maquinaria industrial, para ello los materiales se han dividido en dos bloques.

- Equipo de desarrollo informático: Un ordenador portátil/sobremesa estándar equipado con sistema operativo *Windows*, procesador multinúcleo y memoria RAM suficiente para ejecutar entornos de programación y redes neuronales en la CPU.
- Periféricos de interacción: Una cámara *web* (*webcam*) estándar de resolución básica (720p/1080p) para la captura de vídeo, y la diadema biomédica *NeuroSky MindWave Mobile 2* para la captación de las señales musculares (EMG) mediante conectividad *Bluetooth*.

6.2. Recursos de software y licencias.

La mayor fortaleza económica de este proyecto es que todo está basado en el uso de tecnologías de código abierto (*Open source*) y herramientas de uso libre, lo que reduce el coste de licenciamiento. El cómputo de herramientas de *software* utilizado ha sido:

- Lenguaje y Entorno: Python 3.x (Licencia de código abierto PSF) como lenguaje base, y un entorno de desarrollo integrado (IDE) de uso gratuito como Visual Studio Code o *PyCharm Community*.
- Librerías de Visión e IA: *MediaPipe* de Google (Licencia Apache 2.0) y *OpenCV* (Licencia BSD), ambas gratuitas para uso comercial y de desarrollo.
- Librerías auxiliares y GUI: *CustomTkinter*, *PyAutoGUI* y *PySerial*, todas ellas distribuidas bajo licencias libres y sin coste de adquisición.

- Sistema Operativo: *Windows 10/11* (licencia comercial, ya preinstalada o asumida en el coste del equipo informático del usuario).

6.3. Costes de personal e ingeniería (Cálculo de horas/hombre).

El mayor impacto económico derivado del desarrollo de NEURO-FOCUS se atribuye a la mano de obra especializada. Para estimar este coste, se tomó como referencia el perfil profesional de un Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (DAM) con categoría júnior, estableciendo un coste por hora laboral de 18 euros.

Fase del proyecto	Descripción de la tarea	Estimación de horas	Coste parcial Sobre 18,0€/hora
Fase nº1. Investigación y diseño	Estudio del estado del arte, selección del <i>hardware</i> y diseño de la interfaz y arquitectura	40 horas	720,00€
Fase nº2. Desarrollo y programación	Implementación de <i>Python</i> , integración de <i>MediaPipe</i> , conectividad <i>Bluetooth</i> e hilos de ejecución	80 horas	1440.00€
Fase nº3. Pruebas y depuración	calibración del <i>eye-tracking</i> , ajustes de latencia corrección de errores y pruebas de usabilidad	30 horas	540,00€
Fase nº4 Documentación	Redacción de la memoria técnica, manual de usuario y preparación de la presentación final.	50horas	900,0€
Total		200horas	3600,00€

6.4. Presupuesto total del proyecto.

Para la obtención del presupuesto final se unifican los costes de ingeniería, las licencias de *software* (en este caso con coste nulo gracias al uso de tecnologías *open source*) y los recursos materiales.

Cabe destacar que no se ha imputado el coste de adquisición total del equipo informático de desarrollo ni del terminal móvil empleado como unidad de captura de vídeo. En su lugar, se ha aplicado una cuota de amortización proporcional al periodo de utilización del proyecto, estimada en 50,00 € y 40,00 € respectivamente. Por el contrario, aquellos periféricos adquiridos en exclusiva para el despliegue del sistema sí se han imputado íntegramente en el balance de costes.

Concepto	Descripción	Coste
Costes de personal	200 horas de desarrollo e ingeniería (perfil <i>Junior</i>).	3.600,00 €
Licencias de <i>software</i>	<i>Python, OpenCV, MediaPipe</i> , librerías libres.	0,00 €
Recursos de <i>hardware</i>	Diadema <i>NeuroSky MindWave Mobile 2</i> .	150,00 €
	Amortización del dispositivo móvil / <i>webcam</i>	40,00 €
	Amortización del equipo informático de desarrollo.	50,00 €
Costes indirectos	Suministros básicos (electricidad, internet).	60,00 €
Base Imponible		3.900,00 €
I.V.A. (21%)		819,00 €
PRESUPUESTO TOTAL		4.719,00 €

6.5. Modelo de negocio y viabilidad económica

Aunque la base del proyecto asume unos costes de desarrollo iniciales, la viabilidad a largo plazo de NEURO-FOCUS no se fundamenta en la comercialización de licencias de software propietario. El proyecto preservará su filosofía de código abierto (*Open Source*) con el objetivo de garantizar la accesibilidad universal. Por consiguiente, la estrategia de monetización se articula mediante un modelo B2B (*Business-to-Business*) orientado a clínicas e instituciones sanitarias. Las vías de comercialización planteadas son las siguientes:

Las vías de comercialización planteadas son las siguientes:

Venta del "kit NEURO-FOCUS". Venta de paquetes cerrados que incluirán el equipo informático, la cámara y el sensor de ondas listo para funcionar de manera *plug-and-play* en salas de terapia, aplicando un pequeño margen de beneficio.

-  Soporte técnico e instalación. Cobro de licencias institucionales de manera anual para garantizar un soporte técnico de manera constante junto con una formación al personal de la clínica para su uso con los pacientes.
-  Desarrollos a medida. Facturación por la integración de un api que extrae datos de NEURO-FOCUS para conectarlos con el *software* privado de la institución.

7. Conclusiones y problemas confrontados

7.1. Análisis de resultados: Precisión y tiempo de respuesta.

Tras acabar con la fase de desarrollo e implementación, los resultados que se han podido obtener son que NEURO-FOUCUS confirma la viabilidad de utilizar *hardware* de bajo coste usando una cámara web sencilla y una diadema EEG disponible a nivel comercial para crear un sistema de accesibilidad robusto.

Hablando en términos de tiempo de respuesta o latencia, la arquitectura multihilo ha sido un completo éxito. El hilo que se encarga de la visión artificial apoyándose de la eficiencia de *MediaPipe* procesa la malla facial a una velocidad de entre 30 y 60 fotogramas por segundo, dependiendo de la iluminación en ese momento, lo que da como resultado un refresco del cursor prácticamente casi en tiempo real. Por otra parte, el hilo encargado de la telemetría biométrica captura las señales procedentes de la diadema a una frecuencia de muestreo de 512Hz mediante la biblioteca *PySerial*. Este flujo de alta velocidad permite que el sistema procese y valide un evento de clic en un intervalo de tiempo inferior a los dos milisegundos desde la recepción del impulso bioeléctrico original, garantizando una respuesta casi instantánea.

En cuanto a la precisión, el sistema dota al usuario de una navegación fluida por la interfaz del sistema operativo. La integración del teclado en pantalla junto a la configuración del único clic ha permitido que el usuario pueda escribir textos y navegar sin asistencia externa, cumpliendo el objetivo de otorgar autonomía a personas con movilidad reducida severa.

7.2. Problemas técnicos superados: Ruido lumínico y desviación palpebral.

El desarrollo de sistemas basados en visión artificial y telemetría en tiempo real siempre conlleva enfrentarse a las imperfecciones del mundo físico. Durante el ciclo de vida del proyecto, se identificaron y resolvieron dos bloques técnicos críticos.

El ruido luminoso y vibración del sensor en los inicios. Para corregir el temblor constante del cursor provocado por el ruido lumínico y las fluctuaciones del sensor de imagen (factores que inicialmente impedían realizar clics con precisión), se incorporó un filtro de Kalman. A través de la parametrización de sus matrices de covarianza, el filtro realiza un ponderado óptimo entre la trayectoria predicha y las mediciones ruidosas de la cámara. Este mecanismo eliminó por completo las vibraciones en pantalla, logrando una respuesta de movimiento fluida, estable y con un coste computacional mínimo que no añade retraso al sistema.

El segundo gran obstáculo se basaba en la desviación posicional inducida por el parpadeo. Durante las pruebas de integración, se identificó un defecto ergonómico crítico para la usabilidad: la contracción muscular necesaria para efectuar un parpadeo voluntario alteraba la malla tridimensional de MediaPipe y sus puntos de referencia (*landmarks*). Como consecuencia, el cursor experimentaba desplazamientos abruptos hacia abajo milisegundos antes de consolidar el clic, comprometiendo la viabilidad del *software*. Para mitigar esta anomalía, se diseñó desde cero el mecanismo de retorno temporal (*rollback*). Al mantener un histórico indexado de coordenadas, el software identifica el umbral de deformación facial previo al parpadeo, descarta los datos ruidosos y restablece el cursor a la ubicación estable que ocupaba la mirada con una antelación de 0.5 segundos.

7.3. Desviaciones del anteproyecto:

En cualquier desarrollo de *software*, el anteproyecto sirve como una hoja de ruta inicial. Sin embargo, las pruebas de usabilidad y la integración con el hardware representan los escenarios donde se evalúa la viabilidad real del *software*, obligando a tomar decisiones técnicas complejas. Documentar estas desviaciones en esta memoria no constituye un fracaso en el diseño original, sino que demuestra la capacidad de diagnóstico, adaptabilidad y resolución de problemas del desarrollador ante las demandas del entorno real.

7.3.1. Pivotaje de ondas EEG a señales musculares (EMG) por latencia.

La idea plasmada en el anteproyecto contemplaba el uso de ondas cerebrales puras (midiendo ritmos de concentración y relajación o ritmos de ondas alfa o beta) para accionar el clic del ratón mediante la concentración mental del usuario,

Sin embargo, durante las primeras pruebas de integración fueron descubiertos dos problemas críticos.

El primer desafío técnico fue la elevada latencia de procesamiento. La estabilización de los algoritmos de cálculo cognitivo exigía ventanas temporales de muestreo continuo que generaban un retraso de entre 1 y 3 segundos entre la intención del usuario y la respuesta del sistema. Este desfase imposibilitaba la emulación eficiente del clic, una acción que en entornos informáticos estándar requiere una latencia máxima de 100 milisegundos. La brecha temporal del prototipo inicial, cercana a los 3 segundos, degradaba la experiencia de usuario hasta hacer el sistema inoperativo.

El otro problema era la fatiga cognitiva del usuario, el exigir un esfuerzo de concentración en cada clic del sistema operativo provoca un agotamiento extremo a los pocos minutos del uso, ya que, si para abrir una aplicación necesitas focalizar la mirada durante 3 segundos, eso termina generando fatiga.

La solución técnica fue cambiar el uso del sensor hacia la electromiografía (EMG). Aprovechando que el electrodo frontal de la diadema entra en contacto con el músculo de la ceja, se modificó el analizador para buscar las alteraciones drásticas y los picos de voltaje crudos que provoca un parpadeo voluntario. Al pasar de evaluar ondas cognitivas (EEG) a impulsos musculares (EMG), la latencia se redujo a menos de 2 milisegundos, ofreciendo una respuesta inmediata, nítida y con un coste energético nulo para el usuario a través del algoritmo de "Gatillo Fácil".

7.3.2. Descarte de la versión móvil por densidad de píxeles.

Aunque en las fases iniciales de desarrollo se planteó la idea de un desarrollo multiplataforma donde el código del pc y el móvil fueran independientes, aunque ejecutando la misma lógica, finalmente la versión para dispositivos móviles tuvo que ser descartada por razones de tamaño físico y ópticas .

El problema fundamental nace en la densidad de pixeles (PPI) en relación con el tamaño de una pantalla.

En un monitor de ordenador de 24 pulgadas los elementos de la interfaz como los botones o los menús ocupan un área física grande (pudiendo aumentar la

escala). La cámara *web* dispone de margen de maniobra angular para detectar si el iris va a un botón u otro.

En la pantalla de un teléfono móvil (de unas 6 pulgadas de media), la densidad de píxeles es altísima y los elementos interactivos están extremadamente juntos en un espacio físico microscópico.

La cámara delantera de un *smartphone* no dispone de una densidad óptica ni del ángulo focal que es necesario para distinguir el botón exacto al que el usuario intenta pulsar, por lo que el navegar mediante seguimiento ocular provocaba una cantidad excesiva de clics erróneos, lo que se convertía en una experiencia muy frustrante.

Por mantener la coherencia del producto y su usabilidad se decidió concentrar todo el esfuerzo en el desarrollo en optimizar la versión de escritorio, donde el tamaño físico de las interfaces permite un control milimétrico y verdaderamente útil para el usuario.

7.4. Valoración final del cumplimiento de objetivos.

A pesar de las desviaciones técnicas y los necesarios cambios de rumbo descritos en el apartado anterior, la valoración global del proyecto NEURO-FOCUS es rotundamente positiva. Se ha conseguido desarrollar una aplicación completamente funcional, autónoma y ligera que cumple con el compromiso social e ingenieril planteado en el inicio del grado: romper las barreras de accesibilidad digital.

Haciendo un balance de los objetivos iniciales, se constatan los siguientes hitos:

-  Independencia física: Se ha eliminado la necesidad de utilizar las manos o cualquier tipo de control mecánico para manejar un ordenador.
-  Sustitución del hardware tradicional: El Motor Visual y el Motor de Clic reemplazan con éxito las funciones del ratón físico utilizando componentes comerciales de bajo coste.

- Usabilidad adaptada: Funciones propias como el Freno de Mano inteligente y el Mecanismo de *Rollback* Temporal elevan el *software* desde un prototipo académico a una herramienta de uso diario real, protegiendo al usuario de la fatiga ocular y los falsos positivos.

NEURO-FOCUS demuestra que la combinación de la inteligencia artificial de vanguardia (mallas neuronales de visión artificial) y la telemetría biométrica de consumo no solo es viable, sino que abre un abanico de posibilidades en el desarrollo de interfaces de usuario del futuro, cumpliendo con creces los estándares exigidos para este proyecto de fin de ciclo.

8. Mejoras y trabajos futuros

8.1. Escalabilidad física: Control de silla de ruedas mediante Arduino.

El planteamiento original de NEURO-FOCUS se ha centrado, en su totalidad, en dotar al usuario de un control absoluto sobre el sistema operativo. No obstante, la arquitectura de comunicación y lógica que han sido desarrolladas se podrían perfectamente extrapolar a un entorno físico. Una de las líneas de avance con más peso a futuro es la integración del *software* con plataformas de microcontroladores de código abierto, como puede ser *Arduino*. Estableciendo un canal de comunicación bidireccional, los eventos desencadenados por el seguimiento ocular y la activación muscular podrían traducirse en impulsos eléctricos para gobernar los actuadores y motores de una silla de ruedas.

Esta escalabilidad permitiría transformar una herramienta de accesibilidad informática en un sistema de movilidad integral, dotando al usuario de autonomía física real, utilizando el mismo hardware de control de bajo coste. La implementación de esta mejora supondría el desarrollo de un sistema avanzado de asistencia a la conducción (ADAS) mediante una cámara de 360 grados encargada de monitorizar el entorno. A través de un modelo de *Machine Learning*, el sistema podría comprender el contexto espacial del usuario, valorar el riesgo de colisión y ejecutar de forma automática la detención de la silla de ruedas si se detecta que la maniobra pretendida no es adecuada (por ejemplo, si existe una pared adyacente a la derecha y el usuario intenta avanzar en esa dirección, el *software* emitiría una señal acústica de advertencia).

8.2. Evolución académica: Integración de Machine Learning avanzado (Máster en Tajamar).

La mejora y optimización continua de este sistema informático está directamente vinculada a una próxima etapa de especialización técnica, concretamente a través de la inminente matriculación en el máster en Inteligencia Artificial y *Big Data* en Tajamar. La adquisición de competencias superiores en ciencia de datos permitirá reescribir los algoritmos de estabilización visual aplicando modelos de Machine Learning de última generación desarrollados en el entorno de *Python*. De igual manera, se plantea la reestructuración de la arquitectura local del programa hacia un modelo basado en microservicios alojados en la nube. Mediante el uso de

frameworks de alto rendimiento como *FastAPI*, el sistema podrá procesar la telemetría del usuario de forma remota y devolver los cálculos geométricos en tiempo real, reduciendo al mínimo la carga de procesamiento (CPU) exigida al ordenador local del paciente, aunque habría que tomar en cuenta la latencia para que la experiencia de usuario no perdiera calidad, sino que ganara compatibilidad con ordenadores con muy pocos recursos .

8.3. Aplicación de IA para el análisis de fatiga y atención del usuario.

Tal y como se ha detallado en el desarrollo de la memoria, el sistema actual discrimina y descarta las ondas cerebrales puras (EEG) debido a su elevada latencia, priorizando la señal muscular (EMG) como método de ejecución. Sin embargo, el *hardware* biométrico sigue capturando esta información neurológica. Una línea de investigación futura contempla la integración de algoritmos de Inteligencia Artificial que operen en segundo plano para analizar los ritmos Alfa, Beta y Theta del usuario, cruzando estos datos con la frecuencia de parpadeo monitorizada por la *webcam*. Este análisis cruzado permitiría evaluar en tiempo real la carga cognitiva, el nivel de atención y la fatiga ocular del individuo. Si el modelo detecta agotamiento severo, el software podría adaptar dinámicamente la interfaz (aumentando la tolerancia a fallos, incrementando el tamaño de los elementos gráficos o sugiriendo pausas de descanso preventivas), evolucionando hacia un ecosistema de asistencia verdaderamente inteligente y adaptativo.

9. Bibliografía

- [1] G. Welch y G. Bishop, «An Introduction to the Kalman Filter», 2006. Accedido: 15 de abril de 2026. [En línea]. Disponible en: https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf
- [2] «Guía de detección de puntos de referencia faciales | Google AI Edge», Google AI for Developers. Accedido: 3 de abril de 2026. [En línea]. Disponible en: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker?hl=es-419
- [3] «Kit de herramientas para desarrolladores | Aprende | Intercambia | Publica | Programa para desarrolladores de NeuroSky». Accedido: 3 de abril de 2026. [En línea]. Disponible en: <https://developer.neurosky.com/>
- [4] «¡Bienvenido a la documentación de PyAutoGUI! — Documentación de PyAutoGUI». Accedido: 11 de mayo de 2026. [En línea]. Disponible en: <https://pyautogui.readthedocs.io/en/latest/>
- [5] «Bienvenido a la documentación de pySerial — Documentación de pySerial 3.5». Accedido: 12 de mayo de 2026. [En línea]. Disponible en: <https://pyserial.readthedocs.io/en/latest/>
- [6] «Python 3.14 documentation», Python documentation. Accedido: 21 de mayo de 2026. [En línea]. Disponible en: <https://docs.python.org/3/>
- [7] «ThinkGear Serial Stream Guide », 2006. Accedido: 15 de abril de 2026. [En línea]. Disponible en: <https://cdn.hackaday.io/files/11146476870464/ThinkGearSerialStreamGuide.pdf>

10. Anexos

- 14.1. Manual de usuario y configuración del entorno.(va fuera de aquí)

10.2. Glosario de términos técnicos.

- BCI(*Brain-Computer-Interface*) interfaz cerebro-ordenador tecnología que establece una vía de comunicación ente el cerebro humano y un dispositivo externo permitiendo enviar comandos o señales sin un movimiento físico tradicional.
- CustomTkinter. Librería de *Python* orientada a la creación de interfaces gráficas de usuario(GUI) extiende las capacidades de *Tkinter* permitiendo nuevas funciones.
- EEG (Electroencefalografía): Técnica de monitorización neurológica que registra la actividad eléctrica generada por las neuronas del cerebro (ondas Alfa, Beta, Theta, etc.). En el contexto del proyecto, se captan mediante el sensor frontal de la diadema.
- EMG (Electromiografía): Técnica médica empleada para evaluar y registrar la actividad eléctrica producida por los músculos esqueléticos. En NEURO-FOCUS, los picos de señal EMG generados por los parpadeos forzados o el movimiento de las cejas se utilizan para ejecutar el clic izquierdo debido a su bajísima latencia.
- *Eye-Tracking*(Seguimiento ocular): Proceso tecnológico encarado de medir el punto de mirada o movimiento del ojo respecto a la cabeza.
- *Face mesh o Face Landmarker*: algoritmo de visión artificial proporcionado por *Media pipe* de *Google* estima en tiempo real la geometría de forma tridimensional del rostro humano mediante la proyección de cientos de puntos o *landmarks* sobre el rostro del usuario .

- **Latencia:** Retardo temporal que existe entre la emisión de una orden (por ejemplo, el usuario frunciendo el ceño) y la ejecución visible de la misma en el sistema (el clic del ratón). Minimizar este valor es crítico para la experiencia de usuario.
- **PyInstaller:** Herramienta de despliegue que analiza un código fuente en Python y empaqueta el script junto con todas sus dependencias, librerías dinámicas y el propio intérprete en un único archivo ejecutable o directorio.
- **Filtro de Kalman:** Algoritmo matemático interactivo que funciona mediante una serie de mediciones observadas en tiempo real(no han de ser perfectas pueden contener ruido) con ellas produce una estimación de las variables desconocidas, este *software* se usa para predecir la trayectoria del cursor y la estabilización del cursor producida por el temblor natural del ojo permitiendo una mayor fluidez en el ratón.
- **Rollback (Sistema de retroceso posicional):** lógica programada en el hilo visual de NEURO-FOCUS que funciona como una máquina del tiempo mediante el almacenado de coordenadas en un *buffer* y en el momento de una detección de deformación facial por el clic devuelve el cursor a la última coordenada segura que tenga almacenada normalmente la de hace 0.5 segundos.
- **Standalone:** Termino que define a un programa informático el cual es capaz de funcionar de forma autónoma sin la necesidad de instalar software adicional dependencias o necesidad de conexión a un servidor.
- **Telemetry** o telemetría: proceso automático que efectúa una recopilación de datos en remoto y su transmisión a un sistema de recepción para ser monitorizados. En el proyecto hace referencia al envío constante de paquetes por *bluetooth* desde la diadema al puerto COM

10.2. Fragmentos de código fuente críticos.

10.2.1. Mapeo de coordenadas mediante mínimos cuadrados

El proceso de calibración en NEURO-FOCUS es crítico, ya que la relación entre el movimiento de los globos oculares (espacio bidimensional del rostro) y el puntero en la pantalla (espacio de resolución del monitor) no es lineal ni constante para todos los usuarios. Para resolver esta disparidad, el sistema implementa un modelo de regresión polinómica multivariante resuelto mediante el método de mínimos cuadrados (*Ordinary Least Squares*).

Este enfoque matemático permite transformar la nube de puntos capturada por el algoritmo Face Landmarker de *MediaPipe* hacia un sistema de coordenadas absoluto en píxeles (sin esto sería imposible el control de ratón). La construcción de la matriz de diseño A (se puede observar en el código inferior) incorpora no solo las variables lineales (ex ,ey), sino también términos de interacción (ex .ey) y un término independiente (bias o sesgo), lo que permite al modelo adaptarse a las curvaturas del ojo y a las distorsiones ópticas que introduce la lente de la cámara web. La resolución del sistema sobre determinado garantiza que el error cuadrático medio entre la posición predicha y el objetivo real se minimice, otorgando al sistema una precisión de seguimiento robusta ante las variaciones anatómicas del usuario.

```
def finish_phase1(self):
    """Primera fase del calibrado calculando la matriz de transformación
    mediante el uso del metodo de los minimos cuadrados"""
    self.calibrating_phase1 = False
    #conversion de las muestras a arrays de Numpy para el cálculo matemático
    pts_eye = np.array(self.eye_points)
    pts_screen = np.array(self.screen_targets)
    ex, ey = pts_eye[:, 0], pts_eye[:, 1]
    # constucion de la matriz(A) ajuste polinomico
    # para modelar las no linealidades del seguimiento del ojos
    #se usan las variables ex, ey, el término cruzado (ex*ey) y el sesgo (bias)
    que son:
    # ex: coordenada x del ojo ey: coordenada y del ojo
    # bias: término constante para permitir traslaciones
    # termino curzado: para capturar interacciones entre ex e ey, como cambios en
    la forma del ojo que afectan ambas coordenadas
    A = np.column_stack([ex, ey, ex * ey, np.ones(9)])
    # Resolución del sistema de ecuaciones sobredeterminado A*coef =
    target_screen.
```

```
# Se emplea el método de mínimos cuadrados (least squares) para minimizar el
# error cuadrático medio entre la posición predicha y el objetivo real.
self.coef_x, _, _, _ = np.linalg.lstsq(A, pts_screen[:, 0], rcond=None)
self.coef_y, _, _, _ = np.linalg.lstsq(A, pts_screen[:, 1], rcond=None)
# Transición a la fase 2 para el ajuste de precisión central (offset final)
self.calibrating_phase2 = True
self.sample_buffer = []
self.progress_bar = 0
self.request_mouse_move([screen_w // 2, screen_h // 2])
```

10.2.2. Sistema de Rollback y estabilización mediante Filtro de Kalman

La estabilidad del cursor en el hilo de procesamiento de vídeo se ve comprometida por el ruido óptico y las deformaciones musculares faciales inherentes al parpadeo. Para evitar que estas alteraciones corrompan la telemetría de la mirada y provoquen movimientos involuntarios del ratón, el sistema integra un flujo de estabilización dividido en tres fases concurrentes. A continuación, se presenta la sección de código encargada de orquestar este proceso de filtrado y control de estados:

- Primero, el suavizado de la entrada y la media móvil. Se calcula la distancia entre parpados usando los *landmarks* faciales con los índices 159 y 145, para evitar lecturas erróneas producidos por microparpadeos o un cambio de luz, las muestras están gestionadas con un buffer circular que implementa un algoritmo de tipo FIFO (*Frist in First out*) limitado a 8 datos. La media móvil se calcula con `np.mean` que estabiliza la apertura.
- Segundo, la máquina de estados y el rollback. La aplicación funciona con una máquina de estados que cambia entre "NORMAL" o "DEFORMADO", una vez que se pasa de los umbrales, el cursor se congela y activa el sistema de la máquina del tiempo que recupera la posición almacenada en el (*POSITION_HISTORY*) el cual presenta las coordenadas de hace 0.5 segundos, momento previo antes de la deformación. El puntero se posiciona en esa zona previa para evitar que se produzca una descompensación del filtro de Kalman, por ese cambio, el código reinicia al momento los vectores internos de corrección, (*statePost* y *statePre*) anulando los tirones.

-  Tercero, fase de filtrado estocástico y acotado de bordes (*Clipping*). En condiciones estables y sin fallos, las coordenadas obtenidas tras el mapeo se procesan mediante las ecuaciones de predicción del filtro de Kalman, para eliminar los temblores, finalmente se aplica un truncado geométrico de matrices (*np.clip*) para restringir los píxeles dentro de la resolución de la pantalla, evitando que el cursor se vaya a posiciones fuera de ella. Se calcula la distancia euclídea del desplazamiento para modular la aceleración de manera dinámica.

```
# --- LÓGICA DE CONGELACIÓN con calculo de la distancaia euclidean entre
parpados
eye_openness = math.hypot(lm[159].x - lm[145].x, lm[159].y - lm[145].y)
EYE_HISTORY.append(eye_openness)
if len(EYE_HISTORY) > 8: EYE_HISTORY.pop(0) # bufer ciruclar con algoritmo fido
en 8 muestras
#obtencion de la media movil
avg_openness = float(np.mean(EYE_HISTORY))
# activacion por gestos oculares
if calibrator.is_calibrated:
    # si el ojo se entorna <threshold_close> o se abre mucho > <threshold> se
activa
    if blink_state == "NORMAL" and (
        avg_openness < BLINK_THRESHOLD_CLOSE or avg_openness >
BLINK_WIDEN_THRESHOLD):
        blink_state = "DEFORMADO"
        cursor_frozen = True
        # temporizador de congelacion
        tiempo_descongelacion = time.time() + 1.2

    # --- LA MÁQUINA DEL TIEMPO (ROLLBACK) ---
    # Si tenemos historial, recuperamos la posición de hace ~15 frames (0.5
segundos)
    # Esto deshace la desviación causada por la ceja bajando antes de hacer
clic.
    if len(POSITION_HISTORY) >= 5:
        safe_x, safe_y = POSITION_HISTORY[0] # Cogemos el dato más antiguo
pyautogui.moveTo(safe_x, safe_y) # Teletransporte del ratón
prev_mouse_x, prev_mouse_y = safe_x, safe_y

    # Estabilizamos el predictor de Kalman para que no pegue tirones al
despertar
    medida = np.array([[safe_x], [safe_y]], dtype=np.float32)
    kalman.correct(medida)
    kalman.statePost = np.array([[safe_x], [safe_y], [0], [0]],
dtype=np.float32)
    kalman.statePre = kalman.statePost.copy()
```

```
# Gestion de estados del parpadeo encargado de liberar el cursor
elif blink_state == "DEFORMADO" and (BLINK_THRESHOLD_CLOSE < avg_openness <
BLINK_WIDEN_THRESHOLD):
    blink_state = "NORMAL"
#evaluado del bloqueo del cursor
if blink_state == "NORMAL" and time.time() > tiempo_descongelacion:
    cursor_frozen = False
else:
    cursor_frozen = True
# -----

if calibrator.pending_mouse_move:
    pyautogui.moveTo(calibrator.target_mouse_x, calibrator.target_mouse_y)
    calibrator.pending_mouse_move = False
#flujo de estimaciones y suavizado dinamico
elif calibrator.is_calibrated and not cursor_frozen:
    raw_x, raw_y = calibrator.map_to_screen(rel_eye_x, rel_eye_y)
    medida = np.array([[raw_x], [raw_y]], dtype=np.float32)
    #fase de correccion y prediccion con el filtro de kalman
    kalman.correct(medida)
    pred = kalman.predict()
    # Truncamiento geométrico (clipping) para acotar las coordenadas dentro de la
    resolución útil
    pred_x = float(np.clip(pred[0][0], 0, screen_w - 1))
    pred_y = float(np.clip(pred[1][0], 0, screen_h - 1))
    # Cálculo de la distancia euclidiana del desplazamiento actual para modular
    la aceleracion del cursor
    dist = math.hypot(pred_x - prev_mouse_x, pred_y - prev_mouse_y)
```

10.2.3. Decodificación a bajo nivel de la telemetría Bluetooth

El módulo de la captura biométrica delega la acción del clic a un hilo de ejecución independiente que trabaja de forma asíncrona llamado *DiademaThead*, este se comunica de manera directa con la diadema mediante un puerto serie virtual bajo una conexión Bluetooth, procesando un flujo continuo de bytes.

Para asegurar una latencia nula y minimizar los fallos en la transmisión, el fragmento de código implementa una rutina de análisis (*parsing*) con tres fases.

- Sincronización de tramas y validación El bucle de escucha activa monitoriza el *buffer* de entrada del puerto serie (*in_waiting*). La sincronización de la arquitectura se logra interceptando de manera consecutiva los dos bytes de cabecera definidos por la especificación del fabricante (0xAA 0xAA). Tras validar el inicio de una trama válida, se lee el tercer *byte* que determina la

longitud del paquete (*payload length*). Se incluye un mecanismo de control de flujo recursivo que descarta de inmediato cualquier longitud superior a 170 *bytes*, previniendo errores de desbordamiento de memoria (*buffer overflow*) provocados por tramas corruptas o ruido electromagnético.

- Procesado del *payload* y estación de datos. Una vez se ha extraído el bloque de datos puros y consumido el byte de control (*checksum*), se procesa el paquete. El algoritmo discrimina dinámicamente entre códigos de un solo byte y códigos multi-byte (aquellos superiores a 0x7F). Se extraen selectivamente dos variables críticas: el código 0x02 (*Poor Signal*), que evalúa la conductividad del sensor con la piel del usuario, y el código 0x16 (*Blink Strength*), que mide la magnitud de los picos de tensión electromiográfica (EMG) generados por los músculos faciales
- Control de concurrencia y sistema anti-rebote. Cuando el valor de la señal muscular supera el umbral establecido, el hilo invoca la API del sistema operativo para ejecutar el clic izquierdo del ratón. Debido a que las contracciones musculares sostienen el pico de tensión durante varios milisegundos, el flujo de datos enviará múltiples eventos consecutivos de fuerza elevada. Para evitar la ejecución descontrolada de clics involuntarios, se implementa un filtro de histéresis temporal (anti-rebote por *software*), restringiendo la ejecución de un nuevo comando si no ha transcurrido un delta de tiempo mínimo de 0.8 segundos desde el último evento válido.

```
def run(self):
    """bucle de decodificado de tramas conectado a la diadema en el COM10"""
    try:
        self.diadema = serial.Serial(self.puerto, self.baudios, timeout=1)
        print(f"Diadema conectada en {self.puerto}")
    except Exception as e:
        print(f"Error conectando diadema en {self.puerto}: {e}")
        return

    tiempo_ultimo_clic = 0
    tiempo_señal_perfecta = 0
    #oermanece a la esucha siempre que el puerto serie este abierto
    while self.running and self.diadema.is_open:
        try:
            # comprobar si hay datos de entrada en el puerto serie
```

```

        if self.diadema.in_waiting > 0:
            # Buscar el inicio del paquete (0xAA 0xAA)
            if self.diadema.read() == b'\xaa' and self.diadema.read() ==
b'\xaa':
                payload_len = self.diadema.read()
                if not payload_len: continue
                p_len = ord(payload_len)
                if p_len > 170: continue # borrado de corruptos
                # leer paquete de datos completo y el checksum (1 byte)
encargado de la validacion
                payload = self.diadema.read(p_len)
                self.diadema.read() # checksum

                i = 0
                # analisis de carga en busca de los codigos de calidad de
señal y parpadeo
                while i < p_len:
                    code = payload[i]
                    if code == 0x02: # Calidad de señal
                        sq = payload[i + 1]

                        if sq == 0: # si la señal es cero el contacto sensor
usuario es perfecto

                            tiempo_señal_perfecta = time.time()

                            # HACK SÚPER SENSIBLE
                            if sq > 0 and (time.time() - tiempo_señal_perfecta <
3.0):
                                # gestion del tiempo entre clics
                                if time.time() - tiempo_ultimo_clic > 0.8:
                                    print(f"CLIC HACK (Señal: {sq})")
                                    # pitido
                                    try:
                                        import winsound
                                        winsound.Beep(1000, 150)
                                    except:
                                        pass
                                # ejecucion del clic
                                pyautogui.click()
                                tiempo_ultimo_clic = time.time()
                                tiempo_señal_perfecta = 0 # reinicio
contador

                                i += 2
                            elif code == 0x16: # Parpadeo oficial
                                blink = payload[i + 1]
                                # SENSIBILIDAD MÁXIMA
                                if blink > 15:
                                    if time.time() - tiempo_ultimo_clic > 0.8:
                                        print(f"CLIC OFICIAL (Fuerza: {blink})")
                                        try:

```

```
import winsound
winsound.Beep(1500, 150)
except:
    pass
pyautogui.click()
tiempo_ultimo_clic = time.time()

    i += 2
elif code in [0x04, 0x05]:
    i += 2
elif code == 0x80:
    i += 4
elif code == 0x83:
    i += 25
else:
    #datos irrelevantes
    if code > 0x7F:
        i += 2 + payload[i + 1]
    else:
        i += 2

except Exception:
    pass
#CIERRE SEGURO
if self.diadema and self.diadema.is_open:
    self.diadema.close()
print("Diadema desconectada correctamente.")
```


Agradecimientos

Para empezar este módulo de agradecimientos quiero empezar agradeciendo a mis padres ya que fueron aquellos que me guiaron cuando estaba terminando 4 de la eso para que hiciera el grado medio de SMR ya que si hubiera sido por mi posiblemente ahora estaría en un gimnasio por haber hecho un grado de deporte o en un taller con uno de mecánica o de mecanizado pero por suerte les hice caso renunciando parcialmente a mis gustos de yo quiero trabajar haciendo cosas con mis manos como hace un tornero pero siempre recordare las palabras que me dijo mi padre "en la informática puedes hacer los mismo y ya cuando te metas al superior ni te cuento en vez de estar entre mierda y grasa estas en un ordenador

Para empezar, es de recibo dar las gracias a mis padres. Fueron ellos quienes me guiaron al terminar 4º de la ESO para que estudiara el grado medio de SMR y, posteriormente, DAM. Si hubiera sido por mí, probablemente habría terminado en un taller de mecanizado, buscando trabajar con mis manos. Siempre recordaré las palabras de mi padre: "En la informática puedes hacer lo mismo, creando o arreglando cosas con tus manos, pero en la comodidad de un ordenador en lugar de estar entre grasa". Hoy comprendo que tenía toda la razón. La programación es como ese taller de tornería: en lugar de rebajar una barra de acero para encajar una rosca, picas código desde cero para construir una solución exacta. Gracias por esa visión, por guiarme cuando estaba perdido, por apoyarme en todo momento y por aguantarme en el día a día.

A mi novia, Noelia, le debo gran parte de este logro. Su ayuda y confianza fueron vitales, especialmente en aquel duro final de primer curso donde todo parecía ir cuesta arriba con varias asignaturas pendientes. Sin saber apenas de informática, se las ingenió para pedirle a la Inteligencia Artificial que me generara ejercicios a medida de Bases de Datos y Programación, evaluándome y animándome sin descanso. Ese gesto tan simple fue lo que realmente me dio el empujón para aprobar. También quiero agradecerle su paciencia infinita durante el desarrollo de NEURO-FOCUS: aguantando mis dudas sobre de qué color pintar un botón, corrigiendo mis faltas de ortografía y probando el control de la diadema cientos de veces. Gracias por hacerme mejor persona, por transmitirme tu tranquilidad y por todas esas sonrisas.

Finalmente, el agradecimiento más importante a nivel académico va para mis profesores, las personas que han conseguido que llegue hasta aquí. Entré a DAM con una base muy justa y sin una motivación real, lo cual se reflejó claramente en mi actitud durante el primer año. A pesar de que en muchas clases no presté la atención que debía, los profesores nunca me dejaron de lado. Siempre recordaré la llamada de atención que Graciela nos hizo a Miguel y a mí: "¿Y qué pasa si ahora yo hago como vosotros hacíais y no os hago ni caso?". Por suerte para mí, hizo todo lo contrario. Sus explicaciones me ayudaron a salvar el curso, y esa dedicación docente me hizo cambiar radicalmente de mentalidad en segundo.

Gracias a ellos he pasado de rechazar la programación a encontrar en ella mi verdadera vocación. Aunque al principio este Proyecto Final imponía mucho respeto, ha terminado siendo el trabajo en el que más he disfrutado y aprendido. Cierro esta etapa orgulloso del camino recorrido y con la mirada puesta en mi futuro máster de Inteligencia Artificial en Tajamar. Gracias a todos los que han formado parte del proceso.